Deep Learning for Maritime Vision Applications

Notebooks

Torben Globisch · Chair for Visual and Analytic Computing in Ocean Technologies (VACOT)

Interactive Jupyter/Colab versions of the introductory seminar sessions, written as **show-and-tell** material for the lecture: the presenter runs them top to bottom, there are no exercises. Each notebook mixes short explanations with figures in the style of the slides, **animations** you can step through frame by frame, and live demos on **maritime photos**.

All notebooks, the code and updates on GitHub

github.com/TorbenGl/visionseminar

Scan the QR code or click the link. Every notebook can be opened in Google Colab with one click (no installation needed): see the links below and in the table of notebooks.

In this PDF

Course overview 2

the repository README: all six notebooks, how to use them, how the material was restructured

Notebook 00 · Artificial Neural Networks 5

Session 0 · Part 1 · GitHub · Open in Colab

Notebook 01 · Computer Vision & an MLP on MNIST 21

Session 0 · Part 2 · GitHub · Open in Colab

About this PDF

A static snapshot of the interactive notebooks. The code was run on a CPU (no GPU) on 21 October 2026 to produce the figures and printed outputs, so timings and memory figures differ from a Colab run. Code that only draws figures is hidden, as in the notebooks' collapsed cells. Animations are shown as a few still frames and the neuron playground in its starting position; open the notebooks in Colab for the interactive versions.

Course overview

Notebooks

#	Notebook	Session	Highlights	Open
00	**Artificial Neural Networks**: neurons, activations, learning	0 · Part 1	one running example (iris petals) · neuron playground: decision line + live loss landscape · why non-linearity: a hand-built 2 → 2 → 1 network · gradient-descent animation · backpropagation animation, layer by layer · the training loop step by step on mini-batches of 5 flowers	Open in Colab View on GitHub → in this PDF, p. 5
01	**Computer Vision & an MLP on MNIST**	0 · Part 2	images as numbers · classification / detection / segmentation / depth run live on boat photos · demo: an MLP on MNIST · why MLPs fail on images: shifting a digit by 2–3 px breaks it (animation) · shuffled-pixel experiment · scaling to 224 × 224 × 3: parameters, memory, measured training time, out of memory	Open in Colab View on GitHub → in this PDF, p. 21
02	**Convolutional Neural Networks**	0 · Part 3	mini-project: hand-crafted kernels on the 224 × 224 boat photos → **learning the Sobel kernel** by gradient descent (animation) → CNN on MNIST · animated Sobel walkthrough (all 36 positions) · stride/padding & max-pool animations · translation equivariance animation · parameters at 224 × 224: CNN vs. MLP · **the CNN beats the MLP** with fewer parameters · MLP vs. CNN on shifted digits (animation) · ResNet feature hierarchy	Open in Colab View on GitHub
03	**Vision Foundation Models: the CNN era**	1 · Part 1	short recap: a kernel learned from data (notebook 02) · stride = K → patches · ImageNet history · plain vs. residual networks experiment · ResNet-50 features of **six vessel types** (our maritime dataset): t-SNE, retrieval · **race: linear probe vs. training from scratch** (animation)	Open in Colab View on GitHub
04	**Vision Transformers**	1 · Part 2	self-attention from scratch + animation · patchification animation · "patch embedding = conv" proof · learned positional embeddings · demo: a mini-ViT trained on MNIST · [CLS] attention through 12 layers, and why it looks at the sky (artifact tokens)	Open in Colab View on GitHub
05	**Self-supervised pretraining & transfer**	1 · Part 3	mask-ratio animation + real MAE reconstructions · DINO multi-crop & attention heads · DINOv2 PCA: boat segmentation with zero labels · first-layer filters: random vs. supervised vs. self-supervised · frozen benchmarking on the vessel dataset (k-NN & linear) · **race: linear head vs. partial vs. full fine-tuning** with 50 labels per class (animation)	Open in Colab View on GitHub

The Colab badges point to the `main` branch of this repository. If the repository is private, Colab asks you to authorise GitHub the first time (or use *File → Upload notebook*). For students, either make the repo public or share copies via Google Drive.

Using the notebooks in the lecture

- **GPU:** *Runtime → Change runtime type → T4 GPU*. Notebooks 00 and 02 run fine on CPU; 03–05 really want the GPU.
- **Run everything once before the lecture** (*Runtime → Run all*). This downloads datasets and pretrained weights (up to ~600 MB for notebook 05), so nothing stalls on stage.
- **Animations** render as an HTML player: pause it and use the ◀ ▶ step buttons to walk through e.g. the Sobel computation one kernel position at a time.
- **No exercises:** every notebook is a demo. 🎤 marks prompts for the presenter: questions for the audience (the answer is folded away, click to reveal) or a value worth changing live.
- Figure-drawing code is hidden in collapsed cells (🎨 / 🎞) so the visible code stays focused on the ideas.
- Approximate run time on a Colab T4: 00 ~1 min · 01 ~1.5 min · 02 ~2.5 min · 03 ~2 min · 04 ~2 min · 05 ~3–4 min (03–05 estimated, not yet timed on a T4).

How the material was restructured

The two slide decks were turned into six notebooks, three per session, each with a single theme.

All six notebooks are **show-and-tell**: the former exercises are now demos (the solution code runs in place), and the "try this" prompts became 🎤 questions for the audience with a folded answer.

Session 0 (ANNs & CNNs)

- Notebook 00 follows one running example, the iris petals: a single neuron solved by hand in a playground (Task A works, Task B can't be solved with one line) → a hand-built 2 → 2 → 1 network shows why hidden layers need a non-linearity → gradient descent, backpropagation (layer by layer) and the training loop as animations. The old Iris exercise was removed.
- The **Sobel walkthrough** moved here from the Session 1 recap. It is the clearest intuition for convolution, so it now sits where convolution is introduced (as one animation instead of five slides).
- Notebook 01 makes the case against MLPs on images with three experiments: moving the digits by 2–3 px (accuracy collapses), shuffling the pixels (the MLP doesn't care), and scaling to 224 × 224 × 3 (parameter count, training memory, measured time per step, and an out-of-memory attempt for an MLP with the same shape as the MNIST one).
- Notebook 02 is organised as a mini-project: hand-crafted kernels on the 224 × 224 photos → learn the Sobel kernel from input–target pairs → a CNN on MNIST with fewer parameters, which also survives the shift experiment from notebook 01.
- Stride/padding is taught **once** (notebook 02) instead of in both decks.
- New experiment that ties the session together: an MLP doesn't care if you **shuffle the pixels** (notebook 01), a CNN does (notebook 02). This makes "spatial structure" and "inductive bias" concrete, and it pays off again in the ViT notebook.

Session 1 (Foundation Models & ViT)

- **Running example: a small maritime dataset.** 450 photos of six vessel types (sailboat, speedboat, ferry, cruise ship, cargo/container ship, fishing vessel) from Open Images, with human-verified labels; 50 training and 25 test photos per class. It replaces CIFAR-10 in notebooks 03 and 05, so "learning from data" is shown on maritime data with a realistic number of labels. The image ids are embedded in the notebooks and the photos are downloaded at runtime (≈ 90 MB, cached).
- Learning is shown as it happens: an epoch-by-epoch race between a linear probe and a CNN trained from scratch (03), and between the four adaptation strategies (05). Notebook 05 also shows what random, supervised and self-supervised backbones learned in their very first layer, the bridge back to the Sobel kernel of notebook 02.
- The recap is short and builds the bridge to the new material: one static figure of the kernel that notebook 02 learned from data (same photos, same start; no second animation), then stride = K → patches → tokens.
- Every claim gets a small experiment: skip connections (plain vs. residual at depth 8 and 50), the value of pretraining (linear probe vs. training from scratch), supervised vs. self-supervised features (frozen benchmark), and the adaptation spectrum (linear / MLP head / partial / full fine-tuning).
- Real maritime examples throughout (COCO photos of trawlers, sailboats, lifeboats, ferries, marinas).

Corrections relative to the slides

While converting I fixed a few inconsistencies; you may want to update the PDFs too:

Slide	Issue	In the notebooks
S0 p.16 vs. S1 p.3	"translation **invariance**" vs. "translation **equivariance**"	convolution is *equivariant*; pooling adds (some) *invariance*, shown in an animation (02)
S0 p.16	ANN total "∼234,752" (weights only, no biases) vs. 235,146 on p.12/18; CNN total "∼108,170" mixes counts with and without biases and leaves out the last FC layer	all counts computed from the actual models: MLP 235,146, CNN 108,618
S1 p.16	"Human ≈ 95 %" drawn on a **top-1** accuracy chart; the 5.1 % human figure is a **top-5 error**	ILSVRC top-5 error chart with the human 5.1 % line (03)

Slide	Issue	In the notebooks
S1 p.16	"every winning method has been a CNN (until ViT, 2021)": ILSVRC ended in 2017, ViT never competed	phrased as "from then on every winner was a CNN"
S1 p.11–12	the example input switches from 8×8 to 6×6; the stride-2 slide labels its top row "stride 1"	one consistent 7×7 animation comparing stride 1, stride 2 and padding 1 (02)
S1 p.19	`resnet50(pretrained=True)` is deprecated	`resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)`

Running locally

```
pip install -r requirements.txt
jupyter lab notebooks/
```

Image credits

The iris versicolor photo in notebook 00 is loaded at runtime from Wikimedia Commons, © Danielle Langlois, CC BY-SA 3.0. The maritime example photos are loaded at runtime from the COCO 2017 validation set (Flickr images under Creative Commons licences); they are not redistributed in this repository. The vessel dataset of notebooks 03 and 05 is loaded at runtime from Open Images V7 (annotations CC BY 4.0, photos from Flickr under CC BY 2.0); title, author and Flickr page of every photo are listed in `assets/vessel_photo_credits.csv`. Pretrained models: torchvision, and on the Hugging Face Hub `google/vit-base-patch16-224`, `facebook/vit-mae-base`, `facebook/dino-vits8`, `facebook/dinov2-small`, `depth-anything/Depth-Anything-V2-Small-hf`.

NOTEBOOK 00 · SESSION 0 · PART 1

Notebook 00 · Artificial Neural Networks

Run this notebook

GitHub github.com/TorbenGl/visionseminar/blob/main/notebooks/00_neural_networks_basics.ipynb

Colab **Open in Colab**
colab.research.google.com/github/TorbenGl/visionseminar/blob/main/notebooks/00_neural_networks_basics.ipynb

Static snapshot, executed on a CPU on 21 October 2026. Figure-drawing code is hidden; animations are shown as still frames. Run the notebook in Colab for the interactive version.

DEEP LEARNING FOR MARITIME VISION APPLICATIONS

Session 0 · Part 1 — Artificial Neural Networks

Torben Globisch · Visual and Analytic Computing in Ocean Technologies (VACOT)

Notebook	Topic
→ 00	**Artificial Neural Networks: neurons, layers, learning**
01	Computer Vision & an MLP on MNIST
02	Convolutional Neural Networks
03	Vision Foundation Models: the CNN era
04	Vision Transformers
05	Self-supervised pretraining & transfer

The story of this notebook, told with one small dataset of iris flowers:

1. a single **neuron** draws one straight line, and you can find a good line by hand (🕹️ playground),
2. some problems need more than one line: **hidden layers + non-linear activations**,
3. how a network finds its weights by itself: **loss → gradient descent → backpropagation, layer by layer**,
4. the **training loop**, watched step by step on mini-batches of 5 flowers.

💡 **How to use this notebook:** run the cells top to bottom (`Shift + Enter`), or *Runtime → Run all* before the lecture. This is a show-and-tell notebook: there are no exercises. Figure and animation code is hidden in collapsed cells (🎨 / 🎞️ / 🕹️). Animations come with a player: pause them and step through them frame by frame with ◀️ ▶️.

▸ ⚙️ **Setup: imports, colours & helpers — run me first** *code hidden (setup: imports, colours, helper functions)*

1 · What is an Artificial Neural Network?

Inspired by the brain. The human brain has about 86 billion neurons connected by trillions of synapses. A neuron collects signals through its *dendrites*, integrates them in the *soma* and, if the total is strong enough, *fires* a signal down its *axon*.

An artificial neural network (ANN) copies this structure in a very simplified way:

Biology	ANN
neuron	mathematical node
synapse strength	learnable **weight** w
firing	**activation function** σ

Definition. An ANN is a *parameterised function* $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$, built from layers of neurons, that transforms an input into an output. *Learning* means finding good parameters θ.

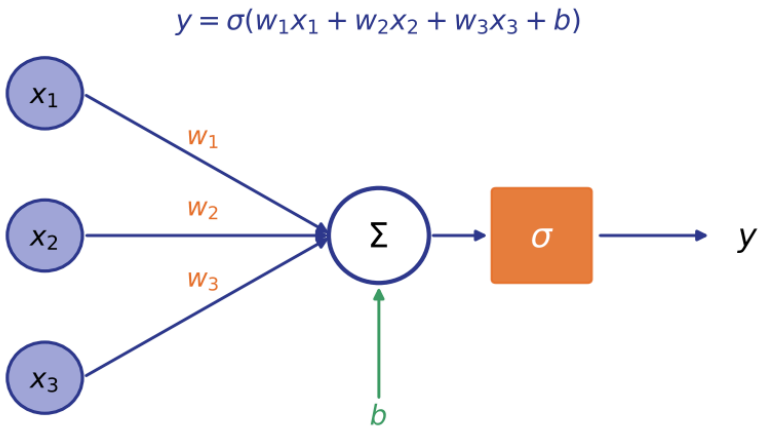
1.1 · The artificial neuron (Rosenblatt's perceptron, 1958)

Each neuron computes a **weighted sum** of its inputs, adds a **bias**, and applies an **activation function**:

$$y = \sigma\Big(\sum_{i=1}^{n} w_i x_i + b\Big) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

- x_i: input features * w_i: weights (*learned*) * b: bias (*learned*) * σ: activation function

▸ 🎨 **Figure: the artificial neuron** *code hidden*



In code, a neuron is one line. Let's compute one by hand:

```
x = torch.tensor([1.0, 2.0, 3.0])      # inputs
w = torch.tensor([0.5, -1.0, 0.25])    # weights
b = torch.tensor(0.1)                  # bias

z = w @ x + b                          # weighted sum: 0.5·1 - 1·2 + 0.25·3 + 0.1
y = torch.relu(z)                      # activation
print(f"z = {z.item():.2f}   →   y = relu(z) = {y.item():.2f}")
```

```
z = -0.65   →   y = relu(z) = 0.00
```

1.2 · Our running example: iris petals 🌸

The *Iris* dataset (Fisher, 1936) contains 150 flowers of three species. For each flower we use two measurements: **petal length** and **petal width**. Our question throughout this notebook is:

Is this flower a *versicolor*? (1 = yes, 0 = no)

We start with the easier half of the problem, **Task A**: tell *versicolor* apart from *virginica*. The neuron gets the two measurements as inputs x_1, x_2 (standardised: 0 = an average flower, ±1 = one standard deviation away) and uses a **sigmoid** $\sigma(z) = \frac{1}{1+e^{-z}}$ as its activation, so its output is a probability:

$$p(\text{versicolor}) = \sigma(w_1 x_1 + w_2 x_2 + b)$$

▸ 📷 What does a *versicolor* actually look like? *code hidden*

Iris versicolor



Photo: Danielle Langlois, CC BY-SA 3.0, via Wikimedia Commons.

```
from sklearn.datasets import load_iris

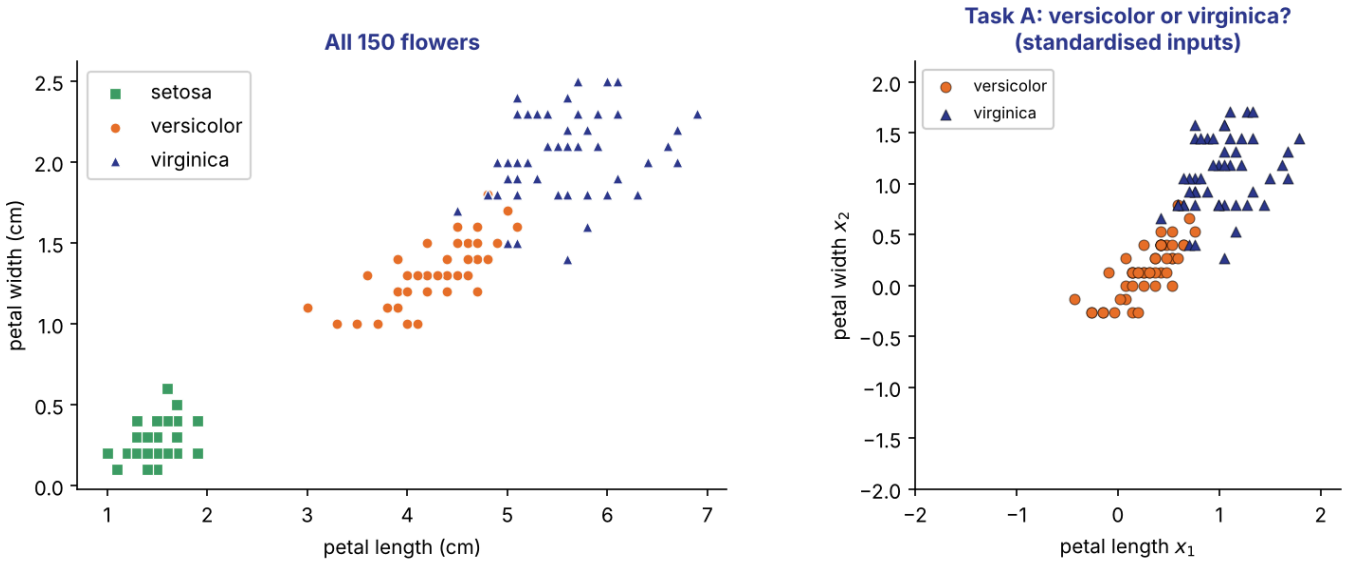
iris = load_iris()
SPECIES = list(iris.target_names)             # ['setosa', 'versicolor', 'virginica']
petals_cm = iris.data[:, 2:4]                 # 150 flowers × (petal length, petal width) in cm
species = iris.target                         # 0 = setosa, 1 = versicolor, 2 = virginica

# The neuron gets standardised inputs: 0 = an average flower, ±1 = one standard deviation away
X_all = ((petals_cm - petals_cm.mean(0)) / petals_cm.std(0)).astype(np.float32)
y_all = (species == 1).astype(np.float32)     # our question: "is it a versicolor?"  (1 = yes, 0 = no)
print("flowers per species:", dict(zip(SPECIES, np.bincount(species).tolist())))
```

```
flowers per species: {np.str_('setosa'): 50, np.str_('versicolor'): 50, np.str_('virginica'): 50}
```

▸ 🎨 Plot helpers for the iris flowers *code hidden*

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
for k in range(3):
    axes[0].scatter(*petals_cm[species == k].T, s=28, c=SP_COLORS[k], marker=SP_MARKERS[k], ec="white", lw=.6,
    label=SPECIES[k])
axes[0].set(xlabel="petal length (cm)", ylabel="petal width (cm)", title="All 150 flowers"); axes[0].legend()
plot_flowers(axes[1], mask=species > 0, legend=True)
axes[1].set_title("Task A: versicolor or virginica?\n(standardised inputs)")
plt.tight_layout(); plt.show()
```



🕹️ Play with a neuron

Three sliders, three parameters: w_1, w_2 and b. Try to make the neuron answer Task A correctly.

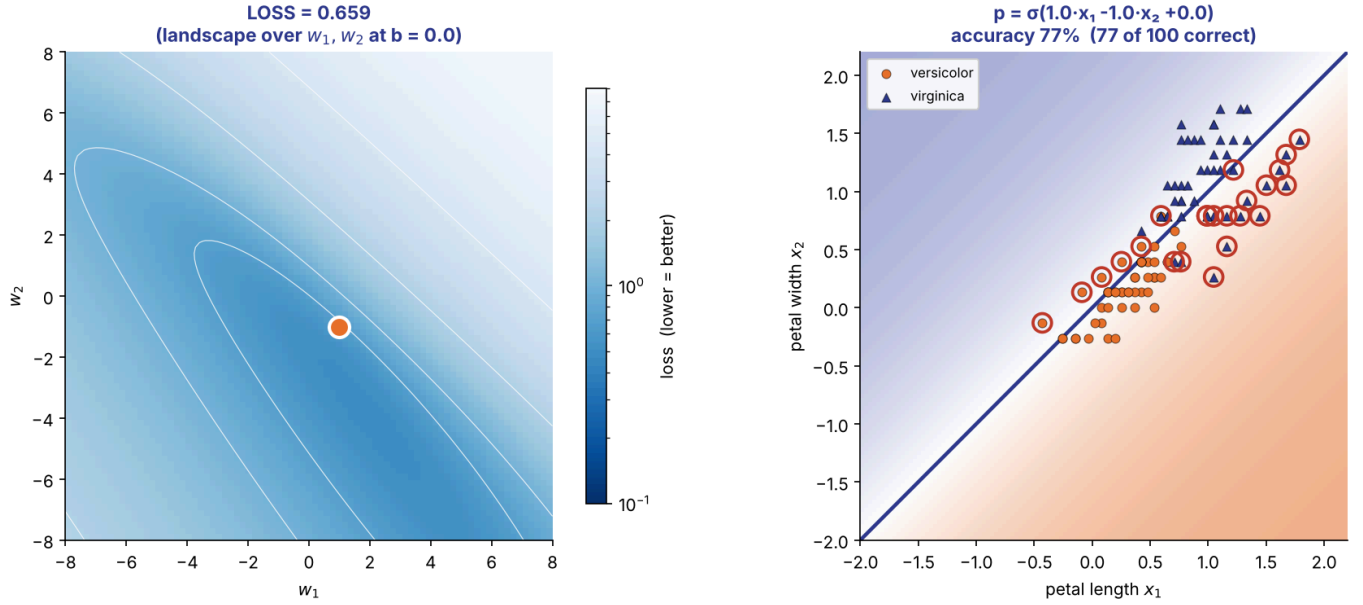
- **Right: the solution.** The background colour is the neuron's answer $p(\text{versicolor})$ at every point; the navy line ($w_1 x_1 + w_2 x_2 + b = 0$, i.e. $p = 0.5$) is its **decision boundary**. Weights **rotate** the line, the bias **shifts** it. Flowers on the wrong side are circled in red.
- **Left: the loss.** One number that says how wrong the neuron is: the average over all flowers of $-\log(\text{probability the neuron gives to the correct answer})$. The map shows the loss for *every* (w_1, w_2) at the current b; the orange dot is you, the trail is where you have been. Dark = low loss = good. **Learning means walking into the dark valley.**

Then switch the dropdown to **Task B** (find the versicolor among *all three* species) and try again.

▸ 🕹️ **Neuron playground (run me, then move the sliders)** *code hidden*

Interactive widget

A dropdown *task* (A: versicolor vs. virginica / B: versicolor vs. setosa + virginica); a slider *w1* (−8 … 8); a slider *w2* (−8 … 8); a slider *b* (−8 … 8). Shown here: the starting position (task = A: versicolor vs. virginica, w1 = 1.0, w2 = −1.0, b = 0.0). Open in Colab to move the sliders.



What did we see?

- **Task A** can be solved *okayish* by hand: around 93 % is possible. A few flowers always stay on the wrong side because the two species overlap; no straight line gets them all. Notice how lower loss on the left goes together with a better line on the right.
- **Task B** cannot be solved by one neuron. *Versicolor* sits **between** setosa (small petals) and virginica (large petals), and a single neuron can only cut the plane into two halves with one straight line. Whatever you do, one of the other species ends up on the versicolor side. The best compromise is about 80 %, and the loss never gets really low.

So we need more than one line. Let's build a small neural network for Task B.

---

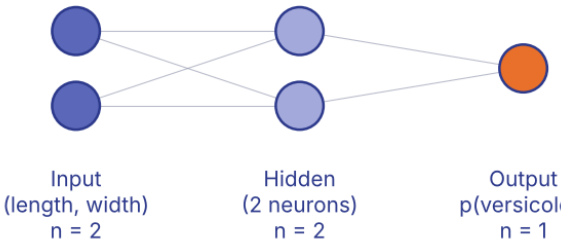
2 · From one neuron to a network: why do we need non-linearity?

Idea: use two neurons, each drawing its own line, and a third neuron that combines their answers:

- hidden neuron 1: *"are the petals bigger than a setosa's?"*
- hidden neuron 2: *"are the petals bigger than a versicolor's?"*
- output neuron: *"yes to neuron 1* ***and*** *no to neuron 2"* → versicolor!

The two neurons in the middle form a **hidden layer**. Together this is a tiny artificial neural network, **2 → 2 → 1**, with $2 \cdot 2 + 2 = 6$ parameters in the hidden layer and $2 \cdot 1 + 1 = 3$ in the output layer: **9 parameters**.

▸ 🎨 **Figure helper: draw a fully-connected network** *code hidden*

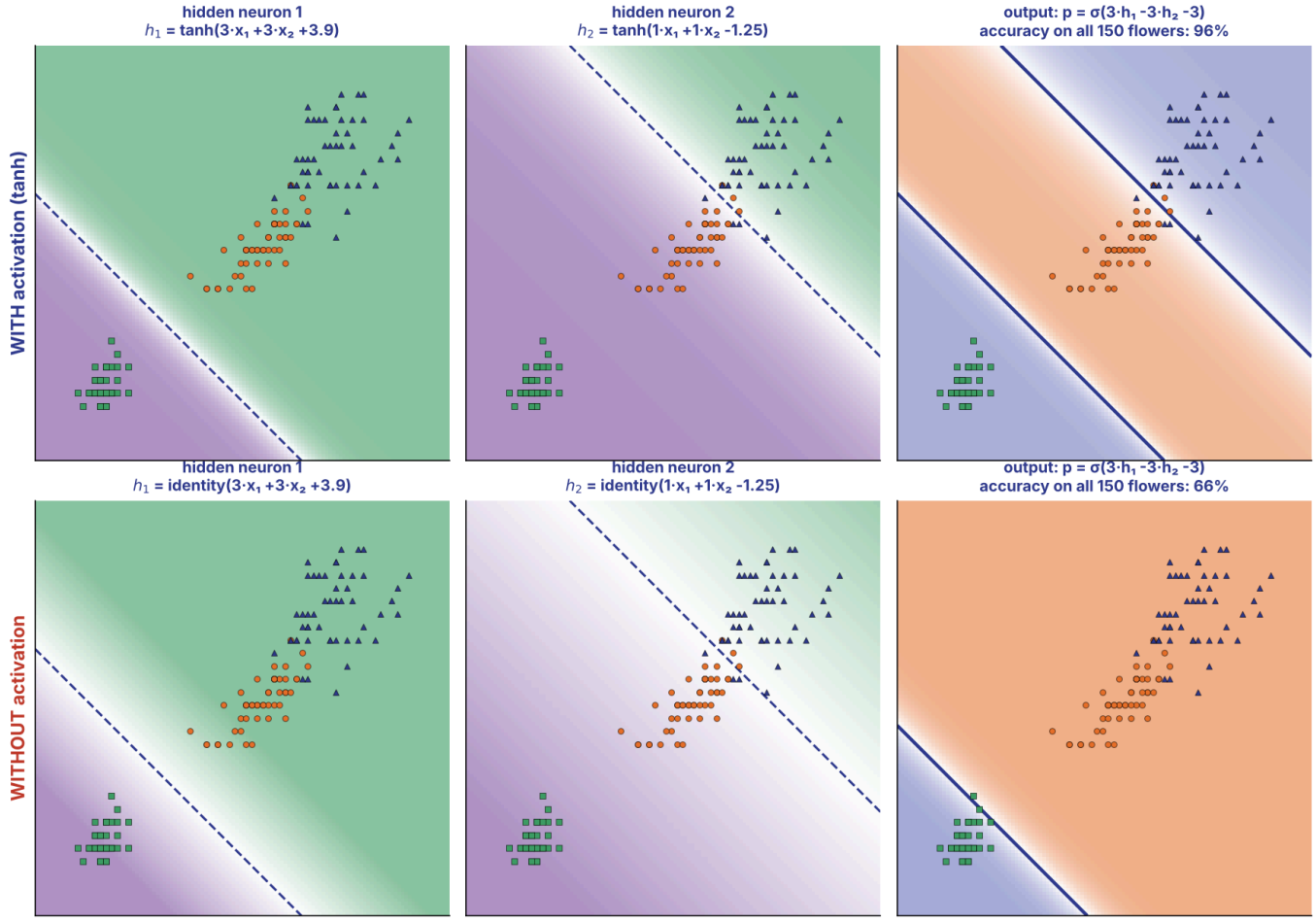


Let's pick the 9 weights **by hand** and look at what every neuron computes. The hidden neurons use the activation $\tanh$ (an S-curve from −1 to +1, like the sigmoid but centred at 0). Top row: **with** the activation. Bottom row: the **same weights without** the activation.

▸ 🎨 **Figure helper: what does every neuron of a 2 → 2 → 1 network compute?** *code hidden*

```
# Hand-picked weights for the 2 → 2 → 1 network
W1 = np.array([[3.0, 3.0],      # hidden neuron 1: "bigger than a setosa?"
               [1.0, 1.0]])     # hidden neuron 2: "bigger than a versicolor?"
b1 = np.array([3.9, -1.25])
w2 = np.array([3.0, -3.0])      # output neuron:   "yes to neuron 1 AND no to neuron 2"
b2 = -3.0

fig, axes = plt.subplots(2, 3, figsize=(12, 8.4))
network_maps(axes[0], W1, b1, w2, b2, np.tanh, "tanh")
network_maps(axes[1], W1, b1, w2, b2, lambda z: z, "identity")
axes[0, 0].set_ylabel("WITH activation (tanh)", fontsize=12, color=NAVY, fontweight="bold")
axes[1, 0].set_ylabel("WITHOUT activation", fontsize=12, color=RED, fontweight="bold")
plt.tight_layout(); plt.show()
```



- **With tanh**, each hidden neuron gives a clean *yes* (+1, green) or *no* (−1, purple) on its side of its line, and the output neuron combines the two answers into a **band**: 96 % correct, far better than any single line.
- **Without an activation**, each hidden "neuron" is just a tilted plane, and adding tilted planes gives another tilted plane. The output collapses back into **one straight line**: we are back to a single neuron.

This is true in general: two linear layers $W_2(W_1x + b_1) + b_2$ are just **one** linear layer $Wx + b$ with $W = W_2W_1$ and $b = W_2b_1 + b_2$. However deep you stack them, without activations you only ever get a straight boundary. A quick numerical check with three layers:

```
torch.manual_seed(0)
deep_linear = nn.Sequential(nn.Linear(2, 16), nn.Linear(16, 16), nn.Linear(16, 2))   # 3 layers, no activation

# Collapse the three layers into a single (W, b)
W = torch.eye(2); b = torch.zeros(2)
for layer in deep_linear:
    W, b = layer.weight @ W, layer.weight @ b + layer.bias

x = torch.randn(5, 2)
print("3-layer network output:\n", deep_linear(x).detach())
print("single linear map  W x + b:\n", (x @ W.T + b).detach())
print("identical:", torch.allclose(deep_linear(x), x @ W.T + b, atol=1e-6))
```

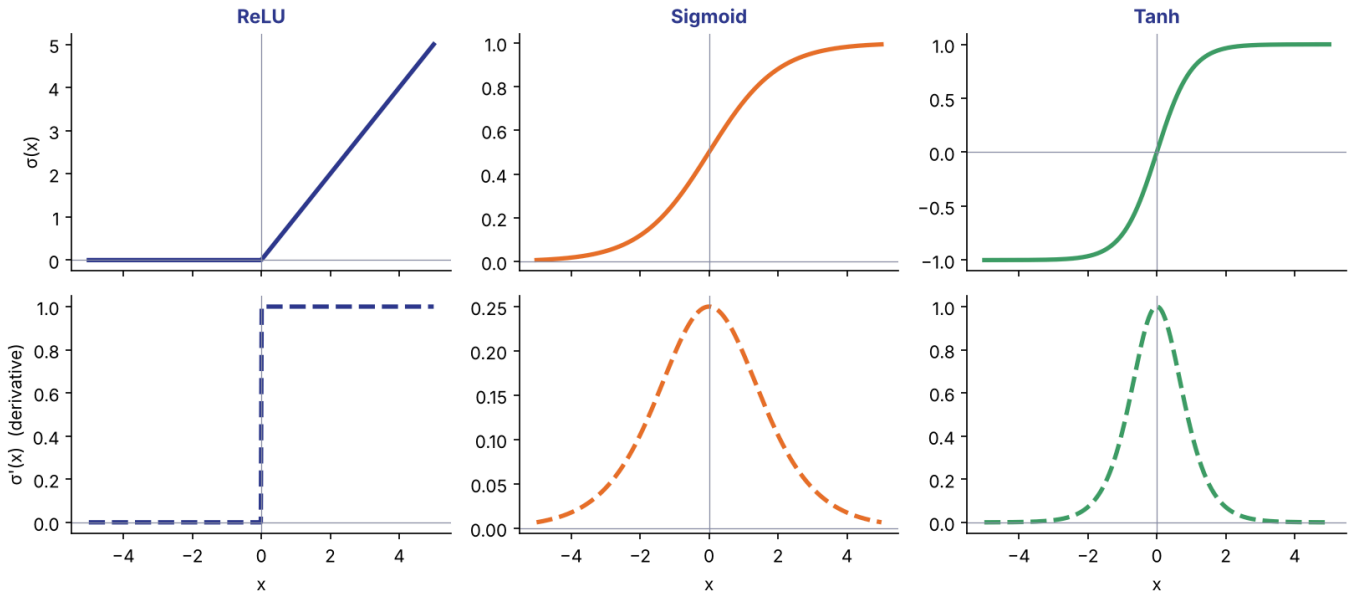
```
3-layer network output:
 tensor([[-0.2901,  0.0164],
        [-0.2253,  0.1882],
        [-0.2947,  0.1503],
        [-0.2272,  0.1870],
        [ 0.0680, -0.4252]])
single linear map  W x + b:
 tensor([[-0.2901,  0.0164],
        [-0.2253,  0.1882],
        [-0.2947,  0.1503],
        [-0.2272,  0.1870],
        [ 0.0680, -0.4252]])
identical: True
```

2.1 · Activation functions

Name	Formula	Notes
ReLU	$\max(0, x)$	default choice in modern (deep) networks: cheap, does not saturate for $x > 0$
Sigmoid	$\frac{1}{1+e^{-x}}$	squashes to $(0, 1)$, used for probabilities (our output neuron)
Tanh	$\tanh(x)$	squashes to $(-1, 1)$, zero-centred (our hidden neurons)

The **derivative** matters as much as the function itself: it decides how much gradient flows back during training (Section 3).

```
xs = torch.linspace(-5, 5, 400, requires_grad=True)
fig, axes = plt.subplots(2, 3, figsize=(11, 5), sharex=True)
for col, (name, fn, color) in enumerate([("ReLU", torch.relu, NAVY), ("Sigmoid", torch.sigmoid, ORANGE), ("Tanh",
    torch.tanh, GREEN)]):
    ys = fn(xs)
    grad, = torch.autograd.grad(ys.sum(), xs)          # derivative via autograd
    axes[0, col].plot(xs.detach(), ys.detach(), color=color, lw=2.5); axes[0, col].set_title(name)
    axes[1, col].plot(xs.detach(), grad, color=color, lw=2.5, ls="--")
    for ax in axes[:, col]: ax.axhline(0, color=GREY, lw=.6); ax.axvline(0, color=GREY, lw=.6)
axes[0, 0].set_ylabel("σ(x)"); axes[1, 0].set_ylabel("σ'(x)  (derivative)")
for ax in axes[1]: ax.set_xlabel("x")
plt.tight_layout(); plt.show()
```



2.2 · Layers and parameters

Real networks have the same structure, just bigger:

- **Input layer**: the raw data (pixel values, sensor readings, petal sizes, ...)
- **Hidden layers**: learn intermediate representations (like our two "bigger than ...?" neurons)
- **Output layer**: the final prediction (e.g. one score per class)

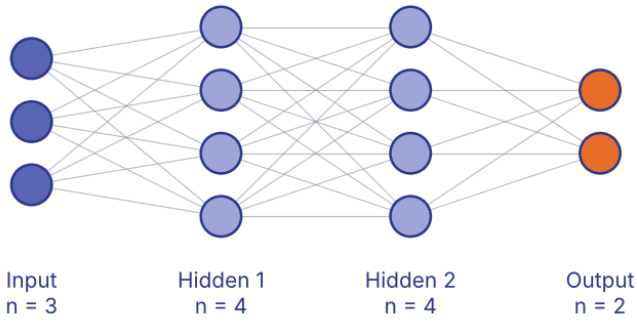
In a **fully-connected (dense) layer** every neuron is connected to every neuron of the next layer. A dense layer with n_{in} inputs and n_{out} outputs has $n_{\text{in}} \cdot n_{\text{out}}$ weights **plus** n_{out} biases. **Hyperparameters** are chosen by *you*, not learned: depth (# layers) · width (neurons per layer) · activation function · learning rate η.

```
fig, ax = plt.subplots(figsize=(6, 3.2)); draw_mlp(ax, [3, 4, 4, 2]); plt.show()

model = nn.Sequential(
    nn.Linear(3, 4), nn.ReLU(),   # input  → hidden 1
    nn.Linear(4, 4), nn.ReLU(),   # hidden 1 → hidden 2
    nn.Linear(4, 2),              # hidden 2 → output
)
print(model)

def count_params(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print("\nparameters per layer:", [(n, p.numel()) for n, p in model.named_parameters()])
print("total:", count_params(model), "= (3·4+4) + (4·4+4) + (4·2+2)")
```




```
Sequential(
  (0): Linear(in_features=3, out_features=4, bias=True)
  (1): ReLU()
  (2): Linear(in_features=4, out_features=4, bias=True)
  (3): ReLU()
  (4): Linear(in_features=4, out_features=2, bias=True)
)
```

```
parameters per layer: [('0.weight', 12), ('0.bias', 4), ('2.weight', 16), ('2.bias', 4), ('4.weight', 8), ('4.bias', 2)]
total: 46 = (3·4+4) + (4·4+4) + (4·2+2)
```

Picking the 9 weights of our iris network by hand already took some thinking. The MLP in the next notebook has 235,146 parameters, modern vision models have millions to billions. **Nobody picks those by hand: the network has to learn them.**

---

3 · How does an ANN learn?

Three steps, repeated over and over:

1. **Forward pass**: the input x flows layer by layer to a prediction $\hat{y}$.
2. **Loss**: one number that measures how wrong the prediction is. You already met it: the left panel of the playground.
3. **Backpropagation + update**: compute $\frac{\partial \mathcal{L}}{\partial w}$ for **every** weight (which way is downhill?), then take a small step downhill: $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$

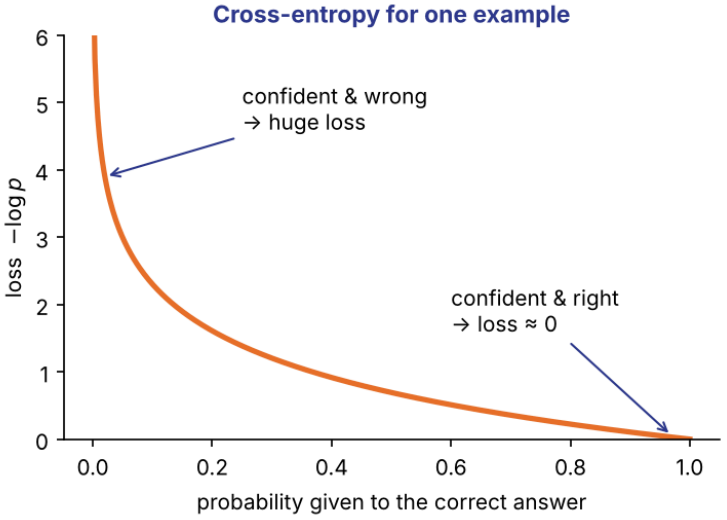
3.1 · The loss

For a yes/no question we use the **binary cross-entropy**, the average over all N examples of the "surprise" about the correct answer:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\Big[\, y_i \log p_i + (1-y_i)\log(1-p_i)\Big]$$

For a versicolor ($y=1$) only $-\log p$ counts, for any other flower only $-\log(1-p)$. Confident mistakes are punished very hard:

```
p = np.linspace(0.001, 1, 400)
fig, ax = plt.subplots(figsize=(5.5, 3.4))
ax.plot(p, -np.log(p), color=ORANGE, lw=2.5)
ax.set(xlabel="probability given to the correct answer", ylabel="loss  $-\\log p$", title="Cross-entropy for one
    example", ylim=(0, 6))
ax.annotate("confident & wrong\n→ huge loss", xy=(0.02, 3.9), xytext=(0.25, 4.6), arrowprops=dict(arrowstyle="->",
    color=NAVY))
ax.annotate("confident & right\n→ loss ≈ 0", xy=(0.97, 0.05), xytext=(0.6, 1.6), arrowprops=dict(arrowstyle="->",
    color=NAVY))
plt.show()
```



3.2 · Gradient descent: let the computer move the sliders

In the playground *you* moved w_1, w_2, b and watched the loss. The **gradient** $\left(\frac{\partial \mathcal{L}}{\partial w_1}, \frac{\partial \mathcal{L}}{\partial w_2}, \frac{\partial \mathcal{L}}{\partial b}\right)$ tells us, for every slider, in which direction the loss goes *up*, so we move each slider a small step in the *opposite* direction. PyTorch computes the gradient for us with `loss.backward()`. Here is the whole algorithm for Task A, starting where the playground started:

```python
XA, yA = torch.tensor(X_all[species > 0]), torch.tensor(y_all[species > 0])   # Task A: 100 flowers

w = torch.tensor([1.0, -1.0], requires_grad=True)       # the playground's starting point
b = torch.tensor(0.0, requires_grad=True)
lr = 3.0                                                # learning rate η: the step size
gd_history = []
for step in range(41):
    p = torch.sigmoid(XA @ w + b)                       # 1. forward pass: the neuron's answer for all 100 flowers
    loss = F.binary_cross_entropy(p, yA)                # 2. loss
    loss.backward()                                     # 3. gradients ∂L/∂w₁, ∂L/∂w₂, ∂L/∂b
    gd_history.append(dict(w=w.tolist(), b=b.item(), grad=w.grad.tolist(), loss=loss.item(),
                           acc=((p > 0.5) == yA).float().mean().item()))
    with torch.no_grad():                               # 4. one step downhill
        w -= lr * w.grad;  b -= lr * b.grad
        w.grad.zero_();    b.grad.zero_()

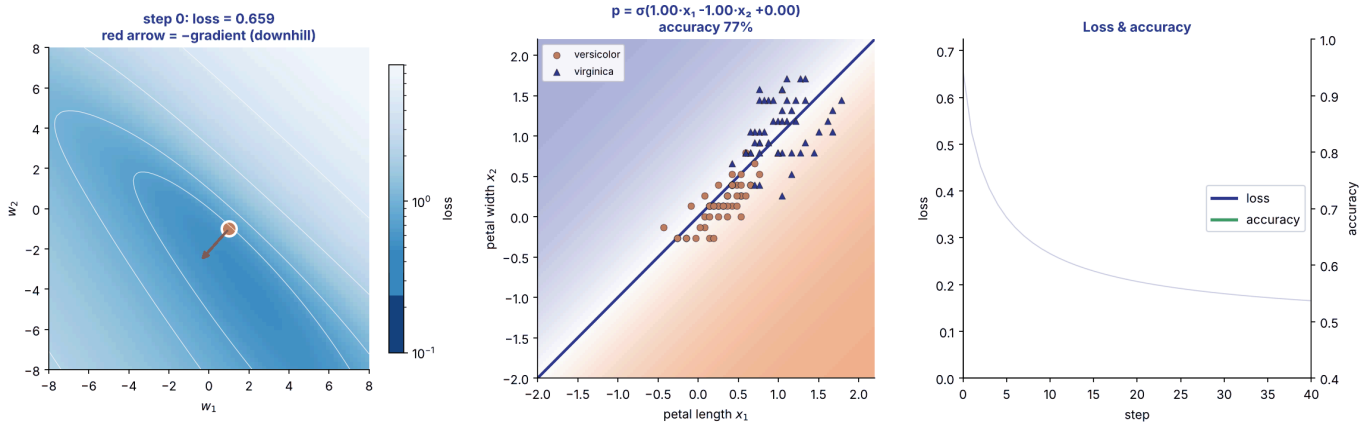
h = gd_history[-1]
print(f"after {step} steps: w = ({h['w'][0]:.2f}, {h['w'][1]:.2f}), b = {h['b']:.2f}  →  loss {h['loss']:.3f},
    accuracy {h['acc']:.0%}")
```

```
after 40 steps: w = (-2.14, -4.46), b = 3.88  →  loss 0.165, accuracy 93%
```

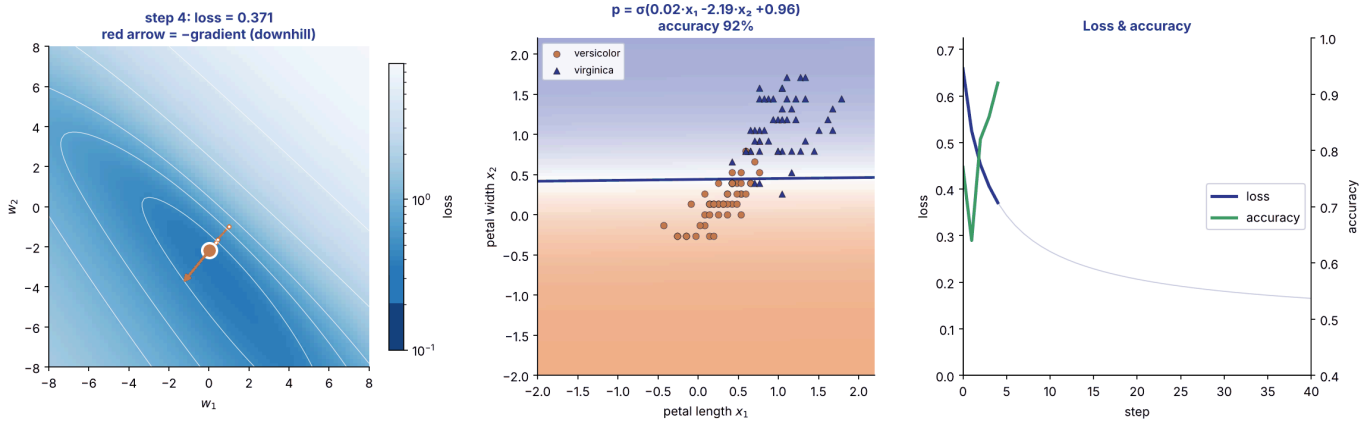
▸ 🎞 **Animation: gradient descent on the neuron playground** *code hidden*

Animation · 41 frames

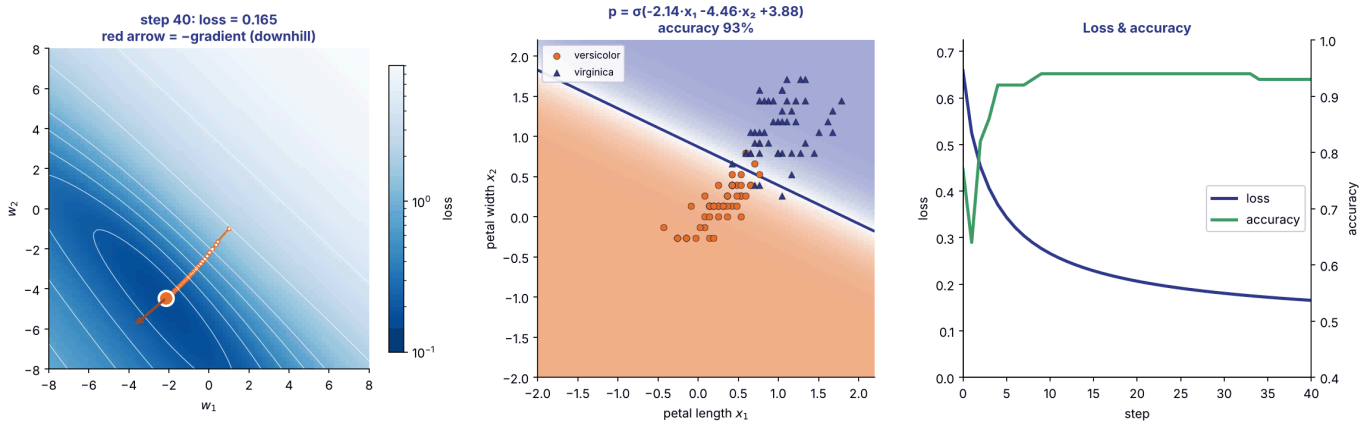
An interactive player in the notebook (play, pause, step frame by frame with ◀ ▶). Shown here: 3 still frames. Open in Colab to step through all of them.



Frame 1 of 41: start: the playground's starting point



Frame 5 of 41: after 4 steps



Frame 41 of 41: after 40 steps: done

Gradient descent does exactly what you did by hand, only systematically: at every step it looks at the slope of the loss landscape (red arrow) and walks downhill. The line on the right swings into place by itself.

The learning rate η sets the step size: here $\eta = 3$. We will see what a bad choice does at the end of the training section.

3.3 · Backpropagation: the chain rule, one layer at a time

For one neuron the gradient is easy. How do we get $\frac{\partial \mathcal{L}}{\partial w}$ for a weight deep inside a network? Take our 2 → 2 → 1 iris network. The forward pass is a chain of simple steps:

$$\underbrace{z_1 = W_1 x + b_1}_{\text{hidden layer}} \;\rightarrow\; h = \tanh(z_1) \;\rightarrow\; \underbrace{z_2 = w_2^\top h + b_2}_{\text{output layer}} \;\rightarrow\; p = \sigma(z_2) \;\rightarrow\; \mathcal{L} = -\log p \;\; (\text{for } y = 1)$$

Backpropagation walks this chain **backwards**. Each layer receives the gradient from the layer above it, multiplies it with its own *local* derivative (chain rule) and passes the result down to the layer below:

backward step	formula
output layer: error signal	$\delta_2 = \frac{\partial \mathcal{L}}{\partial z_2} = p - y$ (sigmoid + cross-entropy simplify nicely)
→ gradients of the output layer	$\frac{\partial \mathcal{L}}{\partial w_2} = \delta_2\, h, \qquad \frac{\partial \mathcal{L}}{\partial b_2} = \delta_2$
hand the signal down	$\frac{\partial \mathcal{L}}{\partial h} = \delta_2\, w_2$
through the activation	$\delta_1 = \frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial h} \odot (1 - h^2) \qquad (\tanh' = 1 - \tanh^2)$
→ gradients of the hidden layer	$\frac{\partial \mathcal{L}}{\partial W_1} = \delta_1\, x^\top, \qquad \frac{\partial \mathcal{L}}{\partial b_1} = \delta_1$

Every layer only needs *its own* inputs and the signal coming from above. That is why it scales to networks with hundreds of layers. The animation runs one forward and one backward pass for a single flower, with real numbers:

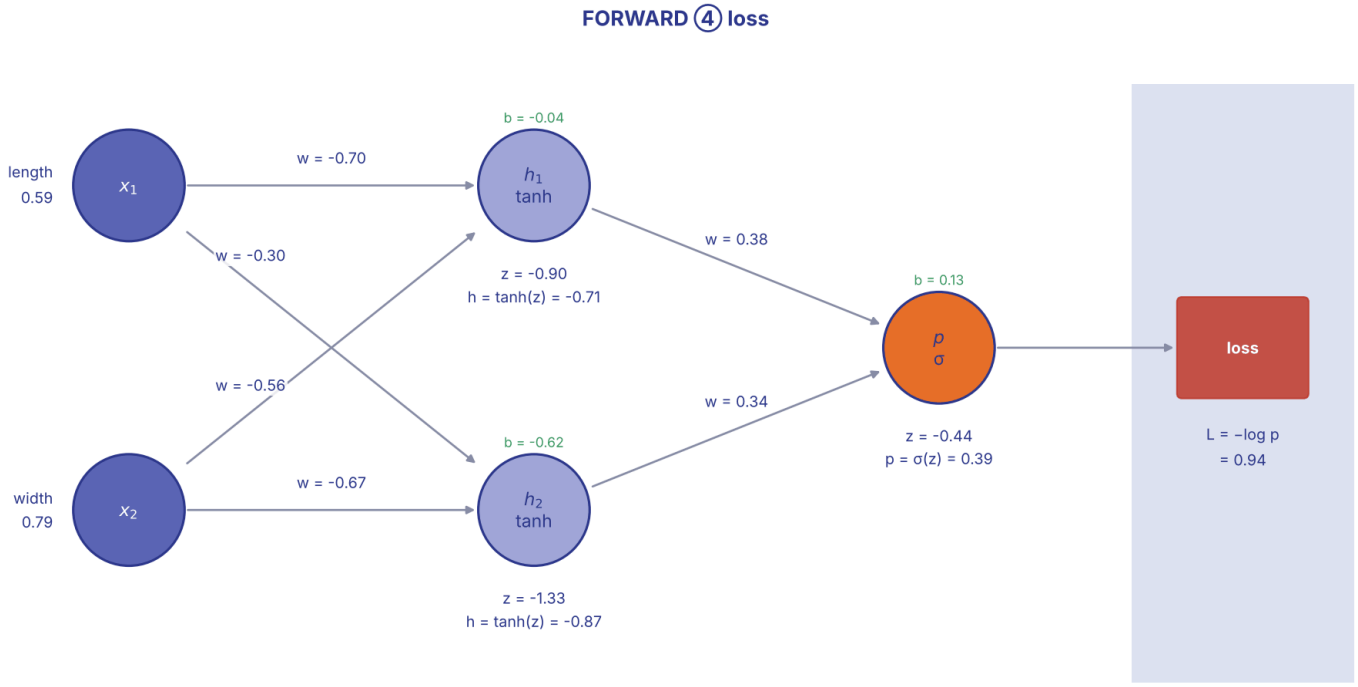
```
SEED = 3                                                  # the same starting weights are used for training below
torch.manual_seed(SEED)
net = nn.Sequential(nn.Linear(2, 2), nn.Tanh(), nn.Linear(2, 1))   # 2 → 2 → 1, random initial weights
FLOWER = 70                                               # one versicolor flower
x0, y0 = torch.tensor(X_all[FLOWER]), torch.tensor(y_all[FLOWER])
print(f"flower #{FLOWER}: {SPECIES[species[FLOWER]]}, petal {petals_cm[FLOWER, 0]} × {petals_cm[FLOWER, 1]} cm"
      f"  →  x = ({x0[0]:.2f}, {x0[1]:.2f}),  y = {y0:.0f}")
```

```
flower #70: versicolor, petal 4.8 × 1.8 cm  →  x = (0.59, 0.79),  y = 1
```

▸ 🎞 **Animation: forward & backward pass through the 2 → 2 → 1 network** *code hidden*

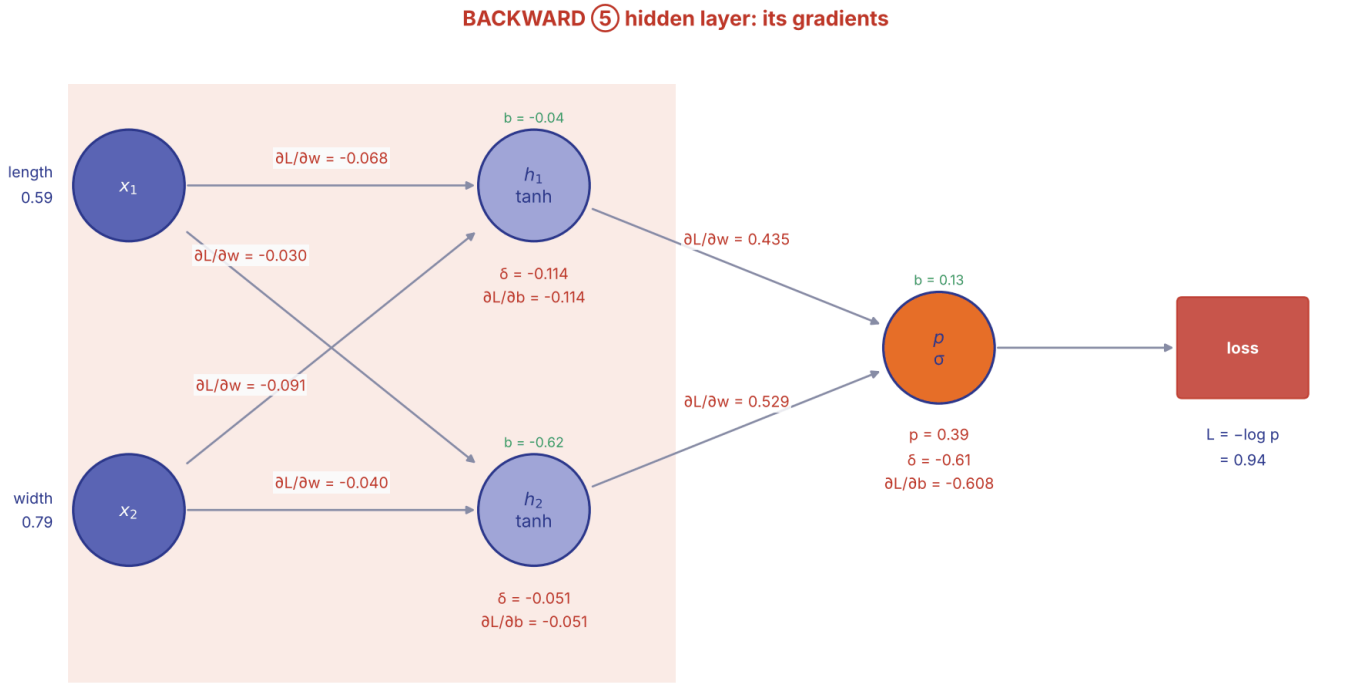
Animation · 11 frames

An interactive player in the notebook (play, pause, step frame by frame with ◀ ▶). Shown here: 3 still frames. Open in Colab to step through all of them.



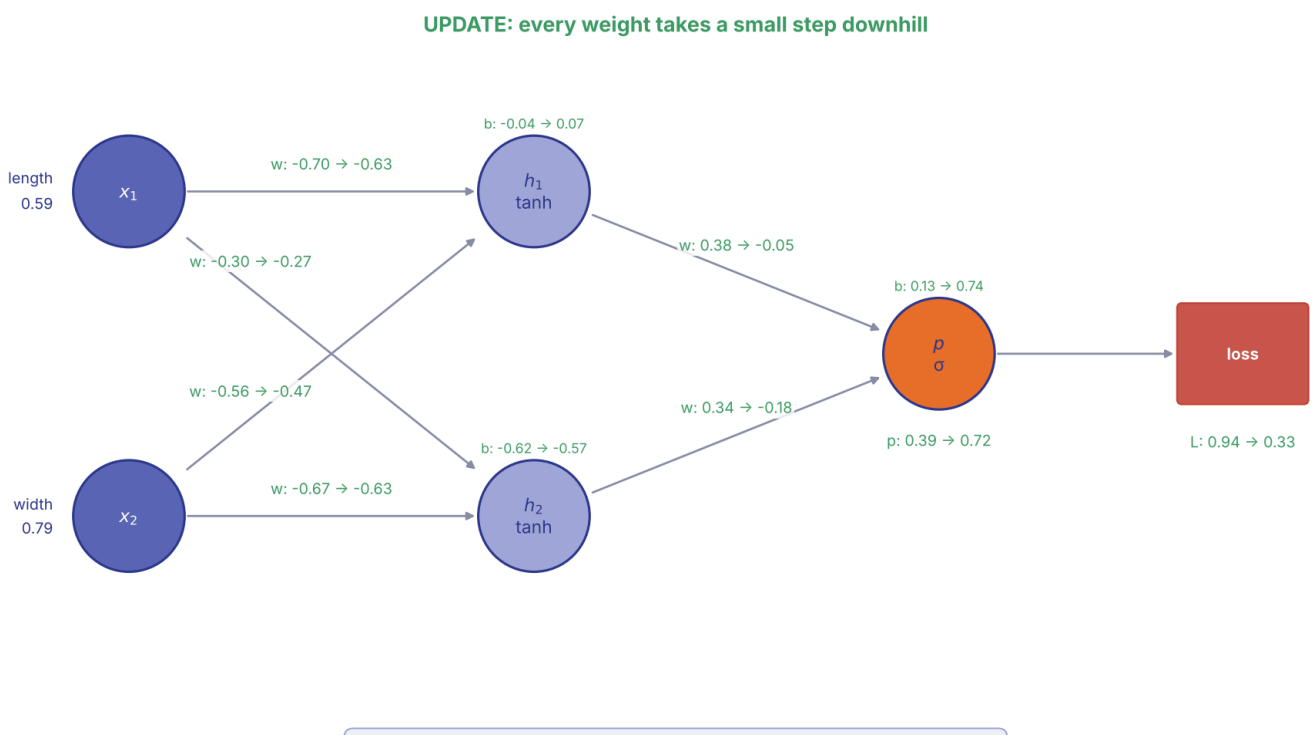
The flower is a versicolor, the network says p = 0.39 → $L = -\log p = 0.94$

Frame 5 of 11: forward pass complete: prediction p and loss L



$\partial L/\partial W_1 = \delta_1 \cdot x$, e.g. -0.114·0.59 = -0.068 $\partial L/\partial b_1 = \delta_1$

Frame 10 of 11: backward pass complete: the gradients of both layers



Frame 11 of 11: update: every weight takes one step downhill

The same computation in code, layer by layer, compared with PyTorch's **autograd** (`loss.backward()`), which does exactly this for any network, automatically:

```
W1_, b1_, w2_, b2_ = net[0].weight, net[0].bias, net[2].weight[0], net[2].bias[0]

# forward, layer by layer
z1 = W1_ @ x0 + b1_;   h = torch.tanh(z1)          # hidden layer
z2 = w2_ @ h + b2_;    p = torch.sigmoid(z2)       # output layer
loss = -(y0 * torch.log(p) + (1 - y0) * torch.log(1 - p))

# backward by hand, layer by layer: output → hidden
with torch.no_grad():
    d_z2 = p - y0                                  # output layer: error signal
    d_w2, d_b2 = d_z2 * h, d_z2                    #   → its gradients
    d_h = d_z2 * w2_                               # hand the signal down
    d_z1 = d_h * (1 - h**2)                        # through tanh
    d_W1, d_b1 = torch.outer(d_z1, x0), d_z1       #   → gradients of the hidden layer

loss.backward()                                    # autograd
for name, mine, auto in [("W1", d_W1, net[0].weight.grad), ("b1", d_b1, net[0].bias.grad),
                         ("w2", d_w2, net[2].weight.grad[0]), ("b2", d_b2, net[2].bias.grad[0])]:
    fmt = lambda t: np.array2string(t.numpy(), precision=4, suppress_small=True).replace("\n", "")
    print(f"{name}   by hand: {fmt(mine):<32} autograd: {fmt(auto)}")
net.zero_grad()
```

```
W1   by hand: [[-0.0678 -0.0905] [-0.0303 -0.0405]] autograd: [[-0.0678 -0.0905] [-0.0303 -0.0405]]
b1   by hand: [-0.1145 -0.0512]                     autograd: [-0.1145 -0.0512]
w2   by hand: [0.4348 0.529 ]                      autograd: [0.4348 0.529 ]
b2   by hand: -0.6085                              autograd: -0.6085
```

3.4 · The training loop

Every training script you will ever see has this skeleton:

```
for epoch in range(num_epochs):          # one epoch = one pass over the whole dataset
    for x, y in train_loader:            # one mini-batch
        y_hat = model(x)                 # 1. forward pass
        loss  = loss_fn(y_hat, y)        # 2. loss
        optimizer.zero_grad()            #    (reset old gradients)
        loss.backward()                  # 3. backpropagation
        optimizer.step()                 # 4. update: w ← w - η ∂L/∂w
```

Instead of computing the loss on all flowers at once, we feed **mini-batches** of 5 random flowers: every step is cheap, and after 30 steps the network has seen all 150 flowers once (one **epoch**). Let's train our 2 → 2 → 1 network on **Task B**, starting from the same random weights as in the backpropagation animation:

```
from torch.utils.data import TensorDataset, DataLoader

torch.manual_seed(SEED)
model = nn.Sequential(nn.Linear(2, 2), nn.Tanh(), nn.Linear(2, 1))      # 2 → 2 → 1, same start as above
X_t, y_t = torch.tensor(X_all), torch.tensor(y_all)
loader = DataLoader(TensorDataset(X_t, y_t, torch.arange(150)), batch_size=5, shuffle=True,
                    generator=torch.Generator().manual_seed(0))         # mini-batches of 5 flowers (+ their index)
loss_fn = nn.BCEWithLogitsLoss()                                        # sigmoid + binary cross-entropy in one
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)                 # plain gradient descent

train_log = []                                                          # remember every step for the animation
for epoch in range(6):
    for xb, yb, idx in loader:
        logits = model(xb).squeeze(1)       # 1. forward pass
        loss = loss_fn(logits, yb)          # 2. loss
        optimizer.zero_grad()
        loss.backward()                     # 3. backpropagation
        train_log.append(dict(idx=idx.numpy(), loss=loss.item(),
                              params=[q.detach().numpy().copy() for q in model.parameters()],
                              grads=[q.grad.numpy().copy() for q in model.parameters()]))
        optimizer.step()                    # 4. update
train_log.append(dict(params=[q.detach().numpy().copy() for q in model.parameters()]))   # final weights

with torch.no_grad():
    acc = ((model(X_t).squeeze(1) > 0).float() == y_t).float().mean()
print(f"{len(train_log) - 1} steps ({epoch + 1} epochs × {len(loader)} mini-batches)  →  accuracy on all 150 flowers:
    {acc:.0%}")
```

```
180 steps (6 epochs × 30 mini-batches)  →  accuracy on all 150 flowers: 96%
```

🎞 Watch the training loop

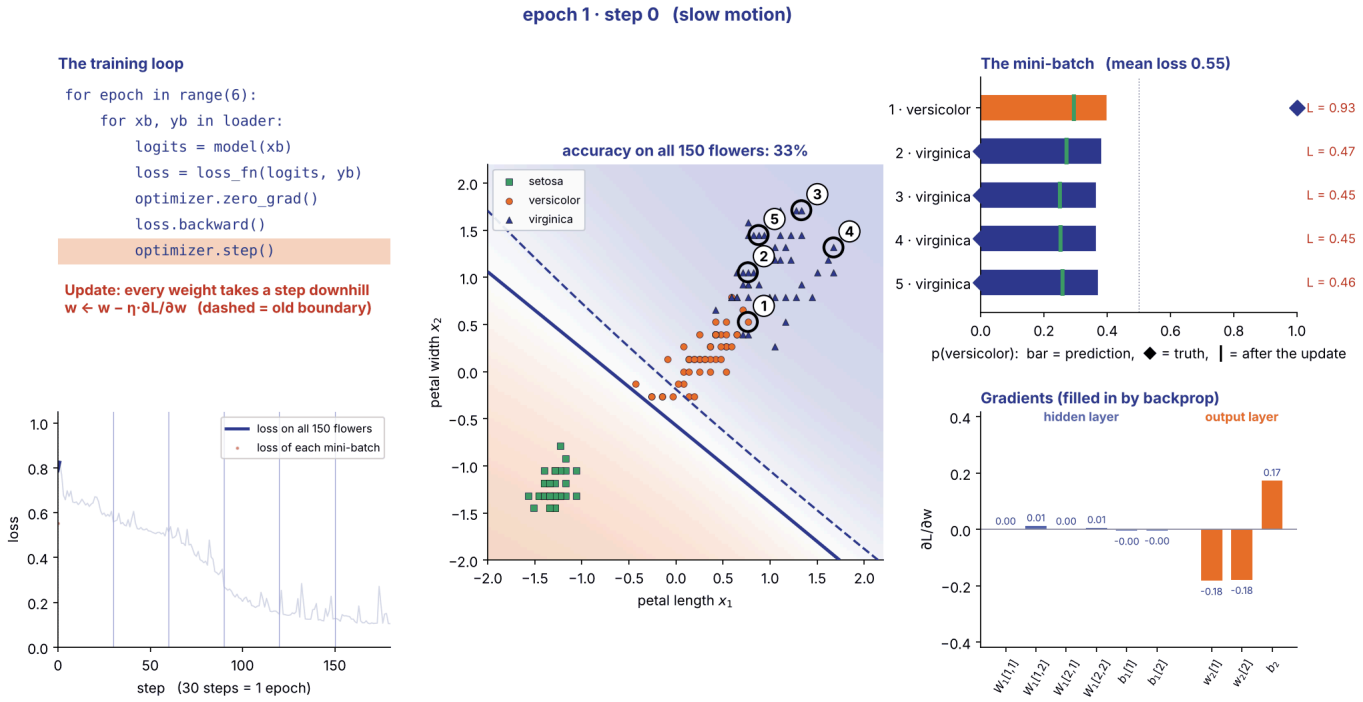
The first **three steps in slow motion**, line by line, then **fast forward** through all six epochs.

- **left:** the training loop (the line being executed is highlighted) and the loss on all 150 flowers
- **middle:** the network's answer everywhere; the 5 flowers of the current mini-batch are enlarged and numbered. During the update the old boundary is shown dashed.
- **right:** the mini-batch (prediction vs. truth and loss per flower) and the 9 gradients, filled in by backpropagation **layer by layer**: output layer first, then the hidden layer.

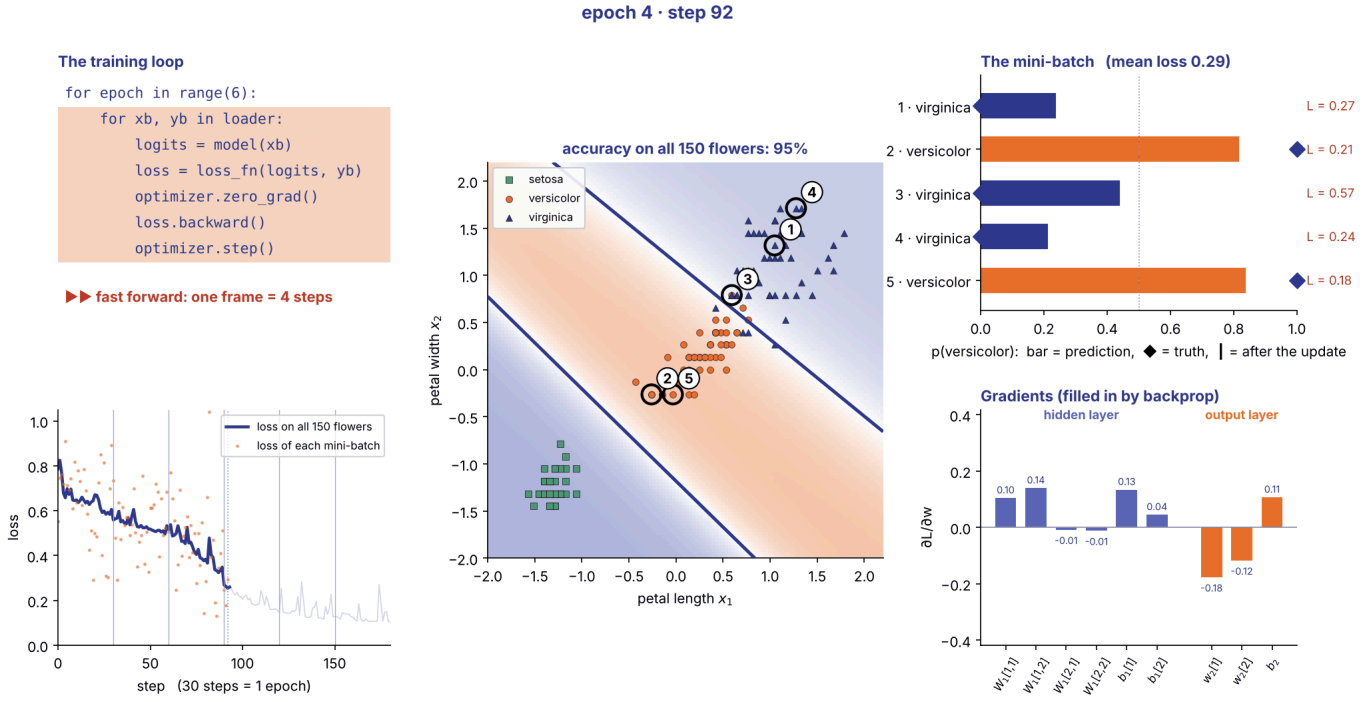
▸ 🎞 **Animation: the training loop, step by step** *code hidden*

Animation · 66 frames

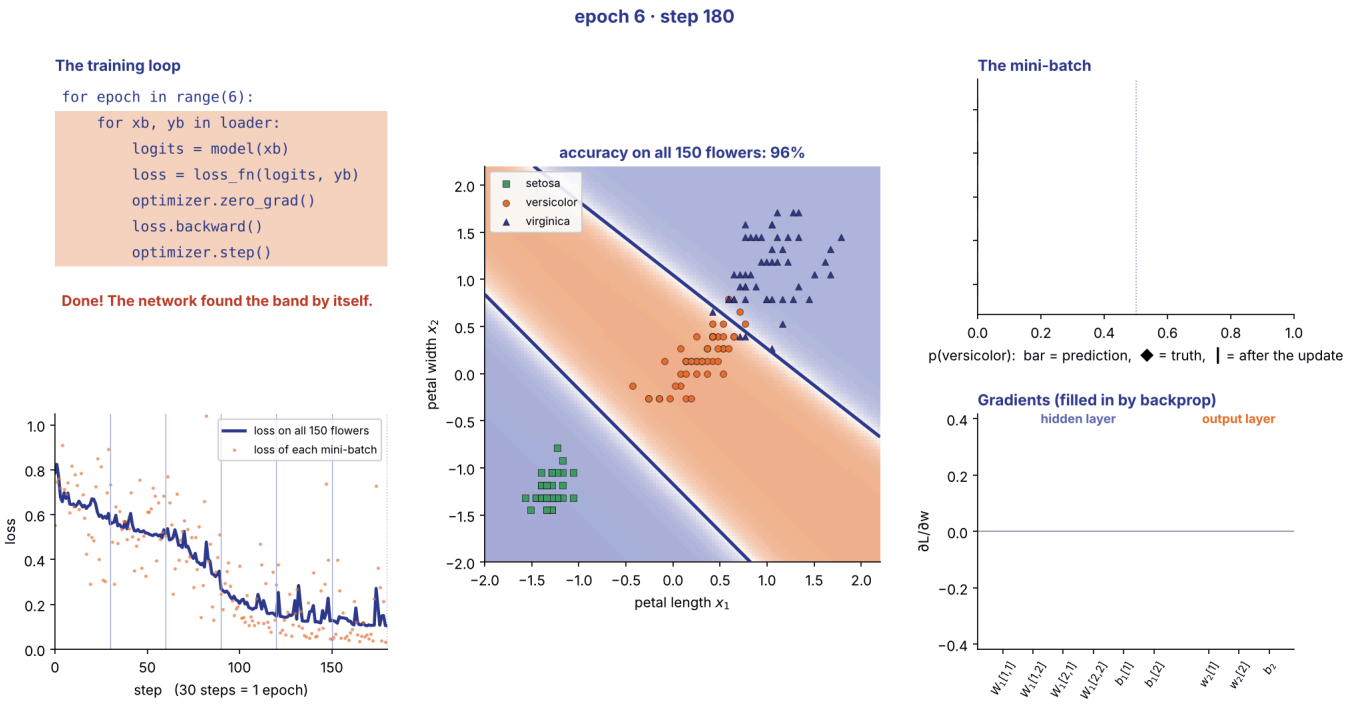
An interactive player in the notebook (play, pause, step frame by frame with ◀ ▶). Shown here: 3 still frames. Open in Colab to step through all of them.



Frame 7 of 66: slow motion, step 0: the update (old boundary dashed)



Frame 44 of 66: fast forward, epoch 4: the band appears



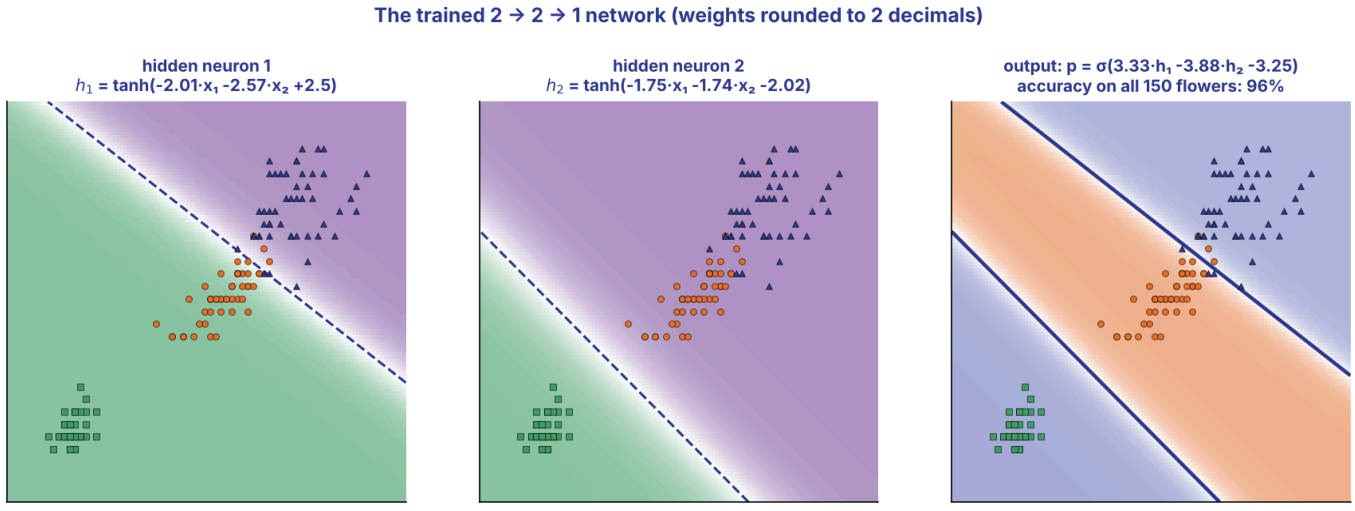
Frame 66 of 66: after 6 epochs: done

Notice the **plateau** in the loss curve: for about two epochs the network answers "not a versicolor" for *every* flower. That is already 67 % correct, because two thirds of the flowers are not versicolors. Then it discovers the band and the loss drops. Also note that single mini-batches are noisy (orange dots): a step can make things worse for some flowers. On average the steps still go downhill.

What did the network learn?

Let's look inside the trained network, exactly like we did with the hand-picked weights in Section 2:

```
W1_t, b1_t, W2_t, b2_t = [q.detach().numpy().astype(float).round(2) for q in model.parameters()]
fig, axes = plt.subplots(1, 3, figsize=(12, 4.4))
network_maps(axes, W1_t, b1_t, W2_t[0], float(b2_t[0]))
fig.suptitle("The trained 2 → 2 → 1 network (weights rounded to 2 decimals)", color=NAVY, fontweight="bold")
plt.tight_layout(rect=(0, 0, 1, 0.92)); plt.show()
```



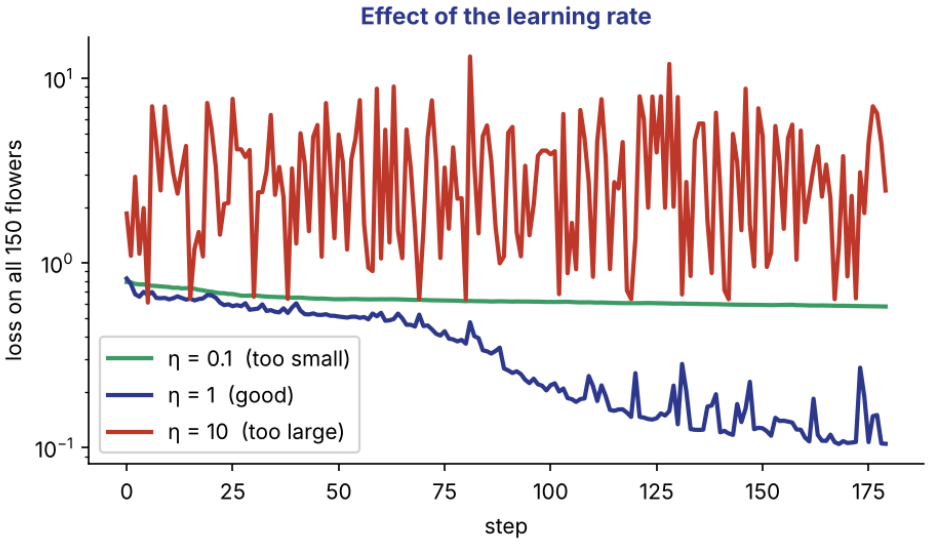
Nobody told the network about "bigger than a setosa" or "bigger than a versicolor". Starting from random weights, gradient descent found **two lines and a way to combine them**, just like our hand-built solution, only by minimising the loss.

The learning rate matters

The same training with three learning rates. Too small: after six epochs the network has barely moved. Too large: every step overshoots, the loss jumps around and the network never settles.

```python
def train_with_lr(lr, epochs=6):
    torch.manual_seed(SEED)
    net = nn.Sequential(nn.Linear(2, 2), nn.Tanh(), nn.Linear(2, 1))
    opt = torch.optim.SGD(net.parameters(), lr=lr)
    loader = DataLoader(TensorDataset(X_t, y_t), batch_size=5, shuffle=True,
    generator=torch.Generator().manual_seed(0))
    losses = []
    for epoch in range(epochs):
        for xb, yb in loader:
            loss = loss_fn(net(xb).squeeze(1), yb)
            opt.zero_grad(); loss.backward(); opt.step()
            with torch.no_grad(): losses.append(loss_fn(net(X_t).squeeze(1), y_t).item())   # loss on all flowers
    return losses

plt.figure(figsize=(7, 3.6))
for lr_, c, note in [(0.1, GREEN, "too small"), (1.0, NAVY, "good"), (10.0, RED, "too large")]:
    plt.plot(train_with_lr(lr_), color=c, lw=2, label=f"η = {lr_:g}  ({note})")
plt.yscale("log"); plt.xlabel("step"); plt.ylabel("loss on all 150 flowers"); plt.title("Effect of the learning
    rate")
plt.legend(); plt.show()
```



Summary

1. A **neuron** computes a weighted sum plus a bias, followed by an activation: $y = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. With two inputs it draws **one straight line**.
2. Problems like Task B need several lines combined: **hidden layers**. Without **non-linear activations** the layers collapse into a single linear map, and we are back to one line.
3. The **loss** measures how wrong the network is; **gradient descent** walks downhill on the loss landscape; the **learning rate** sets the step size.
4. **Backpropagation** computes all gradients with the chain rule, from the output layer back to the input, one layer at a time.
5. The **training loop** repeats: mini-batch → forward pass → loss → zero_grad → backward → step.

Next → 01 · Computer Vision & an MLP on MNIST (p. 21 in this PDF): what happens when the input is an image?

NOTEBOOK 01 · SESSION 0 · PART 2

Notebook 01 · Computer Vision & an MLP on MNIST

Run this notebook

GitHub github.com/TorbenGl/visionseminar/blob/main/notebooks/01_computer_vision_and_mlp_mnist.ipynb

Colab **Open in Colab**
colab.research.google.com/github/TorbenGl/visionseminar/blob/main/notebooks/01_computer_vision_and_mlp_mnist.ipynb

Static snapshot, executed on a CPU on 21 October 2026. Figure-drawing code is hidden; animations are shown as still frames. Run the notebook in Colab for the interactive version.

DEEP LEARNING FOR MARITIME VISION APPLICATIONS

Session 0 · Part 2 — Computer Vision & Your First Image Classifier

Torben Globisch · VACOT

Notebook	Topic
00	Artificial Neural Networks: neurons, layers, learning
→ 01	**Computer Vision & an MLP on MNIST**
02	Convolutional Neural Networks
03	Vision Foundation Models: the CNN era
04	Vision Transformers
05	Self-supervised pretraining & transfer

By the end of this notebook you can

1. explain how a computer "sees" an image (spoiler: as a big array of numbers),
2. name the core computer-vision tasks and what each one outputs,
3. watch a fully-connected network (MLP) learn to classify handwritten digits,
4. show with experiments why an MLP is the wrong tool for images:
   - it is **fragile**: moving a digit by 2–3 pixels breaks it,
   - it **ignores the spatial structure**: shuffling the pixels doesn't bother it,
   - it **does not scale**: at 224 × 224 × 3 it becomes huge, slow, and finally does not fit into memory at all.

💡 Use a GPU runtime: *Runtime → Change runtime type → T4 GPU*. This is a **show-and-tell** notebook for the lecture: run it top to bottom, there are no exercises.

▸ ⚙️ **Setup: imports, colours & helpers — run me first** *code hidden (setup: imports, colours, helper functions)*

1 · What is Computer Vision?

Computer Vision enables machines to interpret and understand visual information (images, videos and 3D data).

Applications: autonomous driving · medical imaging · satellite analysis · quality control · **maritime monitoring** (vessel detection, collision avoidance, debris & oil-spill detection, underwater inspection, …)

1.1 · The machine only sees numbers

Here is a fishing trawler. To the computer it is a $H \times W \times 3$ array of integers in $[0, 255]$: one value per pixel for each of the red, green and blue channels.

```
img = load_image("trawler")
arr = np.array(img)
print("shape:", arr.shape, " dtype:", arr.dtype, f" → {arr.size:,} numbers")

# zoom into a small patch and print its values
y0, x0, s = 274, 159, 8   # a patch on the border between the white stripe and the red hull
patch = arr[y0:y0 + s, x0:x0 + s]

fig = plt.figure(figsize=(15, 4.6))
gs = fig.add_gridspec(1, 3, width_ratios=[1.5, 1, 1.15])
ax = fig.add_subplot(gs[0]); ax.imshow(arr); ax.set_title(f"What we see  ({arr.shape[1]}×{arr.shape[0]} px)")
ax.add_patch(Rectangle((x0, y0), s, s, fill=False, ec=ORANGE, lw=2.5)); ax.axis("off")
ax = fig.add_subplot(gs[1]); ax.imshow(patch, interpolation="nearest"); ax.set_title(f"zoom: {s}×{s} patch");
    ax.axis("off")
ax = fig.add_subplot(gs[2]); gray = patch.mean(2)
ax.imshow(gray, cmap="gray", vmin=0, vmax=255)
for i in range(s):
    for j in range(s):
        ax.text(j, i, int(gray[i, j]), ha="center", va="center", fontsize=8, color="white" if gray[i, j] < 128 else
    "black")
ax.set_title("What the computer sees (grey values)"); ax.axis("off")
plt.tight_layout(); plt.show()
```

```
shape: (335, 500, 3)  dtype: uint8  → 502,500 numbers
```



A colour image is three such grids stacked on top of each other:

```
fig, axes = plt.subplots(1, 4, figsize=(15, 3))
axes[0].imshow(arr); axes[0].set_title("RGB image")
for ax, c, name, cmap in zip(axes[1:], range(3), ["Red", "Green", "Blue"], ["Reds", "Greens", "Blues"]):
    ax.imshow(arr[..., c], cmap=cmap); ax.set_title(f"{name} channel")
for ax in axes: ax.axis("off")
plt.tight_layout(); plt.show()
print(f"A standard 224 × 224 colour input already has 224·224·3 = {224 * 224 * 3:,} numbers.")
```




```
A standard 224 × 224 colour input already has 224·224·3 = 150,528 numbers.
```

1.2 · Why is vision hard?

The same boat looks completely different under a different viewpoint, lighting or scale, and the numbers change completely too. Let's measure how "far apart" images are in raw pixel space:

```
base = np.asarray(load_image("trawler", size=224), dtype=np.float32) / 255
variants = {
    "shifted 10 px": np.roll(base, 10, axis=1),
    "darker (×0.6)": base * 0.6,
    "mirrored": base[:, ::-1],
    "a different boat": np.asarray(load_image("lifeboat", size=224), dtype=np.float32) / 255,
    "random noise": np.random.rand(*base.shape).astype(np.float32),
}
dist = {k: np.sqrt(((v - base) ** 2).mean()) for k, v in variants.items()}

fig, axes = plt.subplots(1, 6, figsize=(16, 3.3))
axes[0].imshow(base); axes[0].set_title("original")
for ax, (k, v) in zip(axes[1:], variants.items()):
    ax.imshow(np.clip(v, 0, 1)); ax.set_title(f"{k}\npixel distance = {dist[k]:.2f}", fontsize=10)
for ax in axes: ax.axis("off")
plt.tight_layout(); plt.show()
```



Shifting the *same* trawler by a few pixels moves it about as far away in pixel space as a picture of a **different boat**! Raw pixel values are a terrible description of *what* is in an image.

Key insight: useful visual features (edges, textures, shapes, parts) must be **learned from data**, not hand-crafted.

---

2 · Core Computer Vision Tasks

Task	Question	Output	Maritime example
Classification	*What* is in the image?	one label per image	ship / no ship
Object detection	*Where* is each object?	bounding boxes + classes	locate vessels in a harbour
Segmentation	*Which pixel* belongs to what?	a label per pixel	water vs. hull vs. debris
Depth estimation	*How far* away is it?	a distance per pixel	3D maps of a scene or the seafloor

Let's run all four on maritime photos. We use off-the-shelf **pretrained** models. You will learn how they work over the next sessions; for now we only look at **what** they output.

(This downloads ~150 MB of model weights; it takes about a minute.)

```
import torchvision
from torchvision.models import ResNet50_Weights
from torchvision.models.detection import fasterrcnn_mobilenet_v3_large_fpn, FasterRCNN_MobileNet_V3_Large_FPN_Weights
from torchvision.models.segmentation import deeplabv3_mobilenet_v3_large, DeepLabV3_MobileNet_V3_Large_Weights
from torchvision.transforms.functional import to_tensor
from transformers import pipeline

img_cls, img_det, img_seg = load_image("trawler"), load_image("marina"), load_image("fishing_boat")

with torch.no_grad():
    # 1) Classification: ResNet-50 trained on ImageNet (1000 classes)
    w = ResNet50_Weights.DEFAULT
    clf = torchvision.models.resnet50(weights=w).eval().to(device)
    probs = clf(w.transforms()(img_cls)[None].to(device)).softmax(1)[0].cpu()
    top = probs.topk(5)

    # 2) Object detection: Faster R-CNN trained on COCO (80 classes)
    w = FasterRCNN_MobileNet_V3_Large_FPN_Weights.DEFAULT
    det = fasterrcnn_mobilenet_v3_large_fpn(weights=w, box_score_thresh=0.75).eval().to(device)
    det_out = det([to_tensor(img_det).to(device)])[0]
    det_names = w.meta["categories"]
```

```
    # 3) Semantic segmentation: DeepLabv3 trained on Pascal VOC classes (21 classes, incl. "boat")
    w = DeepLabV3_MobileNet_V3_Large_Weights.DEFAULT
    seg = deeplabv3_mobilenet_v3_large(weights=w).eval().to(device)
    seg_mask = seg(w.transforms()(img_seg)[None].to(device))["out"].argmax(1)[0].cpu()
    seg_names = w.meta["categories"]

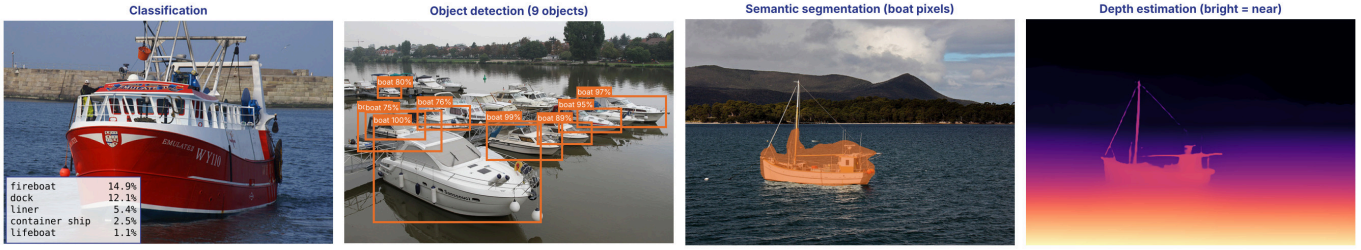
# 4) Monocular depth estimation: Depth Anything V2 (small)
depth = pipeline("depth-estimation", model="depth-anything/Depth-Anything-V2-Small-hf", device=0 if device.type ==
    "cuda" else -1)
depth_map = depth(img_seg)["depth"]

fig, axes = plt.subplots(1, 4, figsize=(18, 4.4))
ax = axes[0]; ax.imshow(img_cls); ax.axis("off"); ax.set_title("Classification")
txt = "\n".join(f"{ResNet50_Weights.DEFAULT.meta['categories'][i]:<16s}{p:6.1%}" for p, i in zip(top.values,
    top.indices))
ax.text(0.02, 0.02, txt, transform=ax.transAxes, family="monospace", fontsize=9, va="bottom", bbox=dict(fc="white",
    alpha=.85, ec=NAVY))

ax = axes[1]; ax.imshow(img_det); ax.axis("off"); ax.set_title(f"Object detection ({len(det_out['boxes'])} objects)")
for box, lbl, sc in zip(det_out["boxes"].cpu(), det_out["labels"].cpu(), det_out["scores"].cpu()):
    x1, y1, x2, y2 = box
    ax.add_patch(Rectangle((x1, y1), x2 - x1, y2 - y1, fill=False, ec=ORANGE, lw=2))
    ax.text(x1, y1 - 3, f"{det_names[lbl]} {sc:.0%}", color="white", fontsize=7, bbox=dict(fc=ORANGE, ec="none",
    pad=1))

ax = axes[2]; seg_small = img_seg.resize(seg_mask.shape[::-1])
ax.imshow(seg_small); ax.axis("off")
mask_rgba = np.zeros((*seg_mask.shape, 4)); mask_rgba[seg_mask == seg_names.index("boat")] =
    mpl.colors.to_rgba(ORANGE, 0.65)
ax.imshow(mask_rgba); ax.set_title("Semantic segmentation (boat pixels)")

ax = axes[3]; ax.imshow(depth_map, cmap="magma"); ax.axis("off"); ax.set_title("Depth estimation (bright = near)")
plt.tight_layout(); plt.show()
```



Note that the classifier has never heard of a "trawler": ImageNet's 1000 classes contain *fireboat*, *lifeboat*, *container ship*, … but no fishing vessels. A model can only predict the labels it was trained on, which is a common problem when we take general-purpose models to the maritime domain.

> **Key observation:** all four tasks share the same core challenge: extracting meaningful **spatial features** from raw pixel arrays. The backbone networks that do this are the topic of the rest of the course.

Today we start with the simplest task, **classification**, on the simplest dataset.

---

3 · Demo: classify handwritten digits with an MLP

Dataset: MNIST

- 60 000 training + 10 000 test images
- 28 × 28 grayscale, digits 0–9

Plan

1. flatten each image: 28 × 28 → 784 inputs
2. an MLP: 784 → 256 → 128 → 10
3. train with cross-entropy (the training loop from notebook 00)
4. look at the training curves
5. look at the predictions, and at the mistakes

3.1 · Load and look at the data

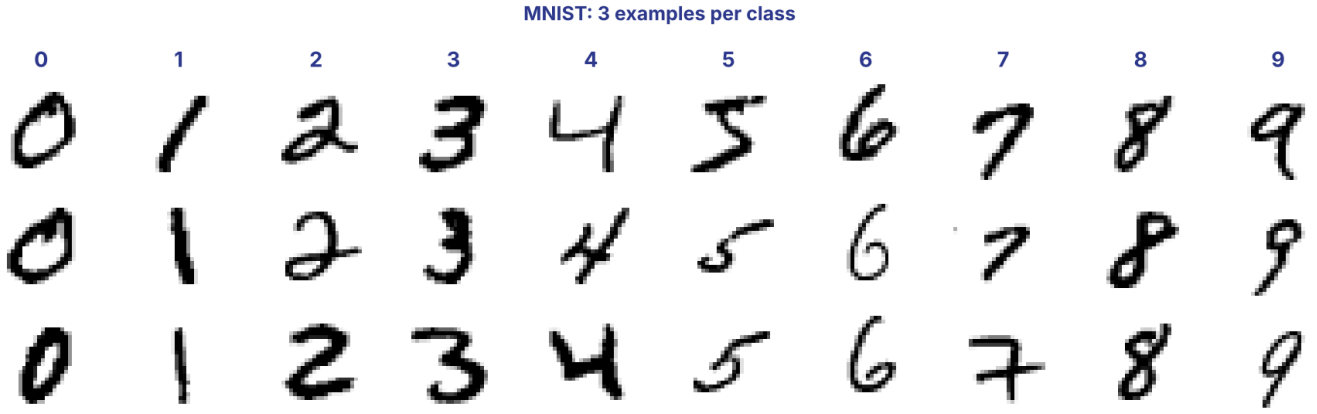
```
from torchvision import datasets
from torch.utils.data import TensorDataset, DataLoader

mnist_train = datasets.MNIST("data", train=True, download=True)
mnist_test = datasets.MNIST("data", train=False, download=True)

# Keep everything as tensors: images in [0, 1], then standardise with the usual MNIST mean/std
MEAN, STD = 0.1307, 0.3081
X_train = ((mnist_train.data.float() / 255) - MEAN) / STD   # (60000, 28, 28)
X_test = ((mnist_test.data.float() / 255) - MEAN) / STD     # (10000, 28, 28)
y_train, y_test = mnist_train.targets, mnist_test.targets
print("train:", tuple(X_train.shape), " test:", tuple(X_test.shape))

fig, axes = plt.subplots(3, 10, figsize=(12, 3.8))
for d in range(10):
    idx = (y_train == d).nonzero()[:3, 0]
    for r, i in enumerate(idx):
        axes[r, d].imshow(mnist_train.data[i], cmap="gray_r"); axes[r, d].axis("off")
    axes[0, d].set_title(str(d), fontsize=13)
plt.suptitle("MNIST: 3 examples per class", color=NAVY, fontweight="bold"); plt.tight_layout(); plt.show()
```

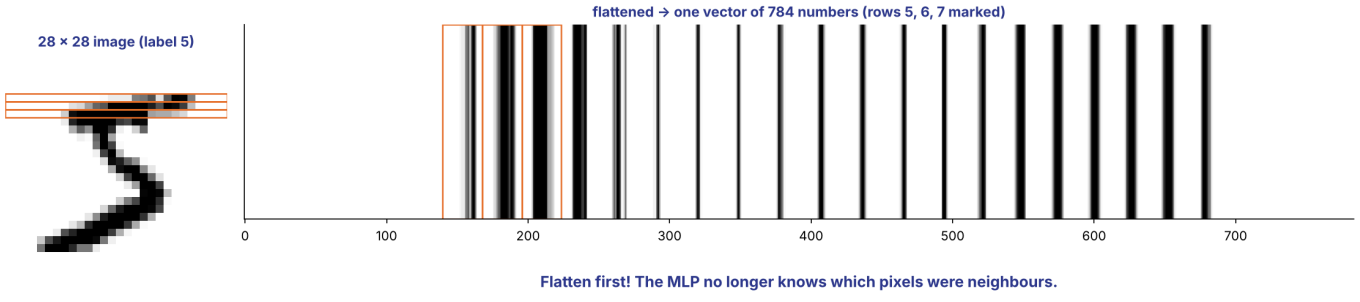
```
train: (60000, 28, 28)  test: (10000, 28, 28)
```



3.2 · Flatten

An MLP takes a **vector** as input, so each 28 × 28 image becomes one long vector of 784 numbers, row after row.

```
digit = mnist_train.data[0].numpy()
fig = plt.figure(figsize=(15, 3.6))
gs = fig.add_gridspec(2, 2, width_ratios=[1, 5], height_ratios=[4, 1])
ax = fig.add_subplot(gs[:, 0]); ax.imshow(digit, cmap="gray_r")
for r in [5, 6, 7]: ax.add_patch(Rectangle((-0.5, r - 0.5), 28, 1, fill=False, ec=ORANGE, lw=1.2))
ax.set_title(f"28 × 28 image (label {y_train[0].item()})"); ax.axis("off")
ax = fig.add_subplot(gs[0, 1]); ax.imshow(digit.reshape(1, -1), cmap="gray_r", aspect="auto")
ax.set_yticks([]); ax.set_title("flattened → one vector of 784 numbers (rows 5, 6, 7 marked)")
for r in [5, 6, 7]: ax.axvspan(r * 28 - 0.5, (r + 1) * 28 - 0.5, fc="none", ec=ORANGE, lw=1.2)
ax = fig.add_subplot(gs[1, 1]); ax.axis("off")
ax.text(0.5, 0.5, "Flatten first! The MLP no longer knows which pixels were neighbours.", ha="center", va="center",
        color=NAVY, fontweight="bold", fontsize=12, transform=ax.transAxes)
plt.tight_layout(); plt.show()
```



3.3 · The training loop

The loop is exactly the one from notebook 00, now with mini-batches of 128 images. We will reuse `fit` in the next notebooks.

```python
def make_loaders(X_tr, y_tr, X_te, y_te, batch_size=128):
    train = DataLoader(TensorDataset(X_tr, y_tr), batch_size=batch_size, shuffle=True)
    test = DataLoader(TensorDataset(X_te, y_te), batch_size=1000)
    return train, test

@torch.no_grad()
def evaluate(model, loader):
    model.eval(); loss, correct, n = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        loss += F.cross_entropy(logits, y, reduction="sum").item()
        correct += (logits.argmax(1) == y).sum().item(); n += len(y)
    return loss / n, correct / n

def fit(model, train_loader, test_loader, epochs=5, lr=1e-3):
    model.to(device)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    hist = {"train_loss": [], "test_loss": [], "test_acc": [], "step_loss": []}
    for epoch in range(epochs):
        model.train(); t0, running = time.time(), 0.0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            loss = F.cross_entropy(model(x), y)      # forward + loss
            opt.zero_grad(); loss.backward(); opt.step()   # backprop + update
            running += loss.item() * len(y); hist["step_loss"].append(loss.item())
        test_loss, test_acc = evaluate(model, test_loader)
        hist["train_loss"].append(running / len(train_loader.dataset))
        hist["test_loss"].append(test_loss); hist["test_acc"].append(test_acc)
        print(f"epoch {epoch + 1}/{epochs}  train loss {hist['train_loss'][-1]:.4f}  "
              f"test loss {test_loss:.4f}  test acc {test_acc:.2%}  ({time.time() - t0:.1f}s)")
    return hist

def count_params(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

train_loader, test_loader = make_loaders(X_train, y_train, X_test, y_test)
```

3.4 · The network: 784 → 256 → 128 → 10

`nn.Flatten()` turns every 28 × 28 image into a vector of 784 numbers, followed by three dense layers with ReLU in between.

```python
def build_mlp():
    return nn.Sequential(
        nn.Flatten(),                       # (B, 28, 28) → (B, 784)
        nn.Linear(784, 256), nn.ReLU(),
        nn.Linear(256, 128), nn.ReLU(),
        nn.Linear(128, 10),                 # one logit per digit
    )
```

```python
mlp = build_mlp()
print(mlp)
print(f"\nparameters: {count_params(mlp):,}   (784·256+256) + (256·128+128) + (128·10+10) = "
      f"{(784*256+256) + (256*128+128) + (128*10+10):,}")
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): ReLU()
  (3): Linear(in_features=256, out_features=128, bias=True)
  (4): ReLU()
  (5): Linear(in_features=128, out_features=10, bias=True)
)

parameters: 235,146   (784·256+256) + (256·128+128) + (128·10+10) = 235,146
```

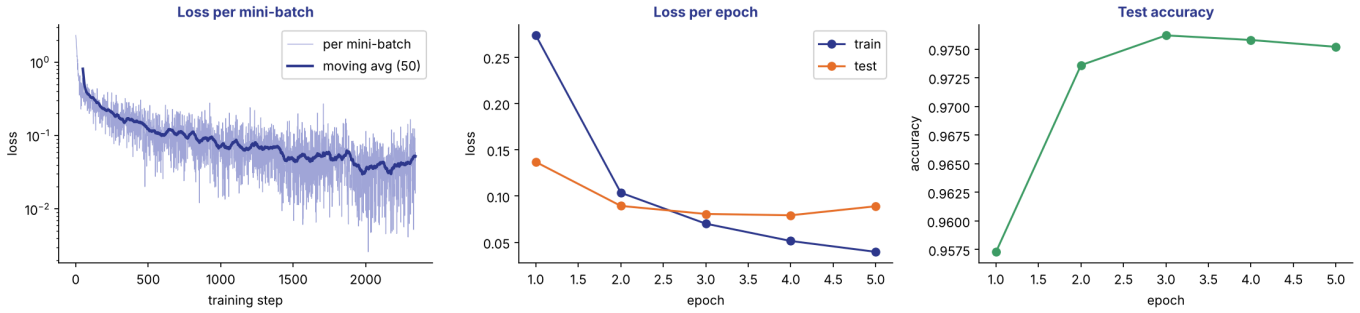
3.5 · Train

```
torch.manual_seed(0)
mlp = build_mlp()
hist_mlp = fit(mlp, train_loader, test_loader, epochs=5)
```

```
epoch 1/5  train loss 0.2737  test loss 0.1368  test acc 95.73%  (1.4s)
epoch 2/5  train loss 0.1036  test loss 0.0893  test acc 97.36%  (1.4s)
epoch 3/5  train loss 0.0701  test loss 0.0806  test acc 97.62%  (1.5s)
epoch 4/5  train loss 0.0514  test loss 0.0792  test acc 97.58%  (1.4s)
epoch 5/5  train loss 0.0397  test loss 0.0889  test acc 97.52%  (1.5s)
```

3.6 · Training curves

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6))
axes[0].plot(hist_mlp["step_loss"], color=LAVENDER, lw=.6, label="per mini-batch")
k = 50; smooth = np.convolve(hist_mlp["step_loss"], np.ones(k) / k, mode="valid")
axes[0].plot(range(k - 1, len(hist_mlp["step_loss"])), smooth, color=NAVY, lw=2, label=f"moving avg ({k})")
axes[0].set(xlabel="training step", ylabel="loss", title="Loss per mini-batch", yscale="log"); axes[0].legend()
ep = range(1, len(hist_mlp["train_loss"]) + 1)
axes[1].plot(ep, hist_mlp["train_loss"], "-o", label="train"); axes[1].plot(ep, hist_mlp["test_loss"], "-o",
    label="test")
axes[1].set(xlabel="epoch", ylabel="loss", title="Loss per epoch"); axes[1].legend()
axes[2].plot(ep, hist_mlp["test_acc"], "-o", color=GREEN)
axes[2].set(xlabel="epoch", ylabel="accuracy", title="Test accuracy")
plt.tight_layout(); plt.show()
```



3.7 · Look at the predictions

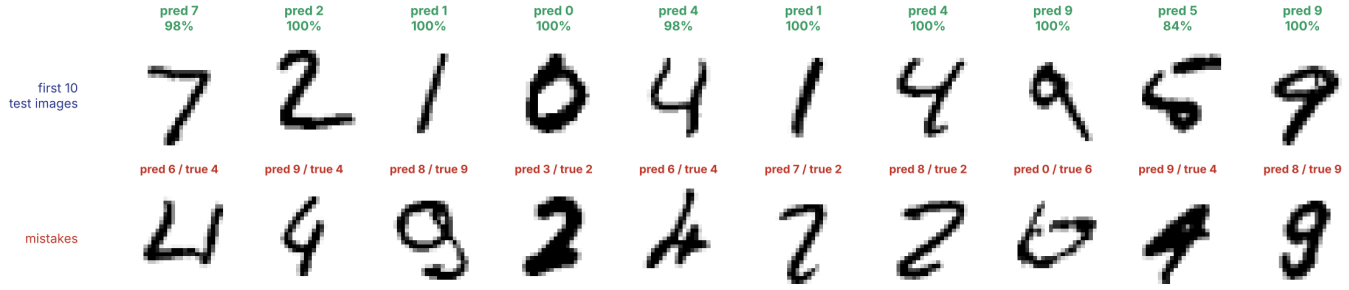
Accuracy alone hides a lot. Look at individual predictions, the confusion matrix and the **mistakes**.

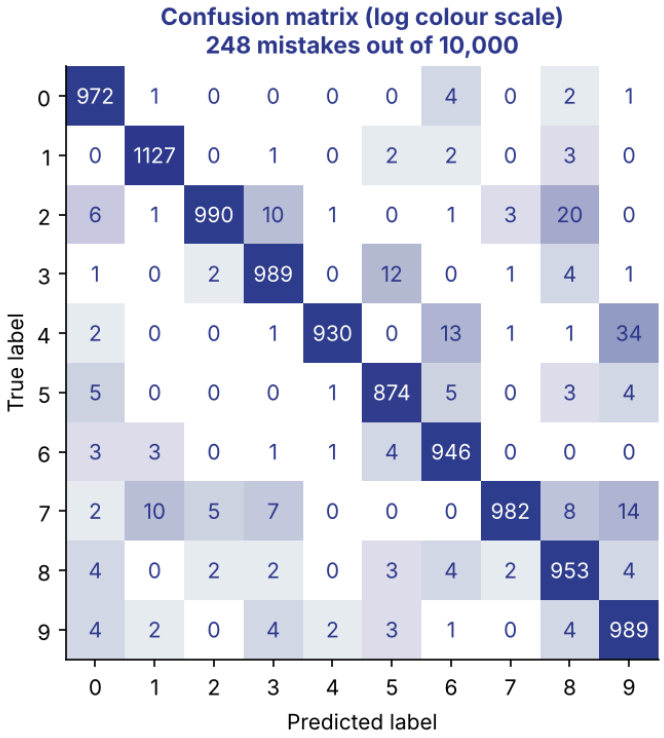
```
@torch.no_grad()
def predict(model, X):
    model.eval()
    return torch.cat([model(X[i:i + 1000].to(device)).softmax(1).cpu() for i in range(0, len(X), 1000)])

probs_mlp = predict(mlp, X_test); pred_mlp = probs_mlp.argmax(1)

fig, axes = plt.subplots(2, 10, figsize=(15, 3.6))
for k, ax in enumerate(axes[0]):
    ax.imshow(mnist_test.data[k], cmap="gray_r"); ax.axis("off")
    ok = pred_mlp[k] == y_test[k]
    ax.set_title(f"pred {pred_mlp[k].item()}\n{probs_mlp[k].max():.0%}", color=GREEN if ok else RED, fontsize=10)
wrong = (pred_mlp != y_test).nonzero()[:, 0]
for ax, k in zip(axes[1], wrong[:10]):
    ax.imshow(mnist_test.data[k], cmap="gray_r"); ax.axis("off")
    ax.set_title(f"pred {pred_mlp[k].item()} / true {y_test[k].item()}", color=RED, fontsize=9)
axes[0, 0].text(-12, 14, "first 10\ntest images", fontsize=10, ha="right", va="center", color=NAVY)
axes[1, 0].text(-12, 14, "mistakes", fontsize=10, ha="right", va="center", color=RED)
plt.tight_layout(); plt.show()

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from matplotlib.colors import LinearSegmentedColormap, LogNorm
cm = confusion_matrix(y_test, pred_mlp)
fig, ax = plt.subplots(figsize=(5.5, 5))
ConfusionMatrixDisplay(cm).plot(ax=ax, cmap=LinearSegmentedColormap.from_list("navy", ["white", NAVY]),
    colorbar=False,
                                im_kw=dict(norm=LogNorm(vmin=1, vmax=cm.max())))
ax.set_title(f"Confusion matrix (log colour scale)\n{len(wrong)} mistakes out of {len(y_test):,}"); plt.show()
```

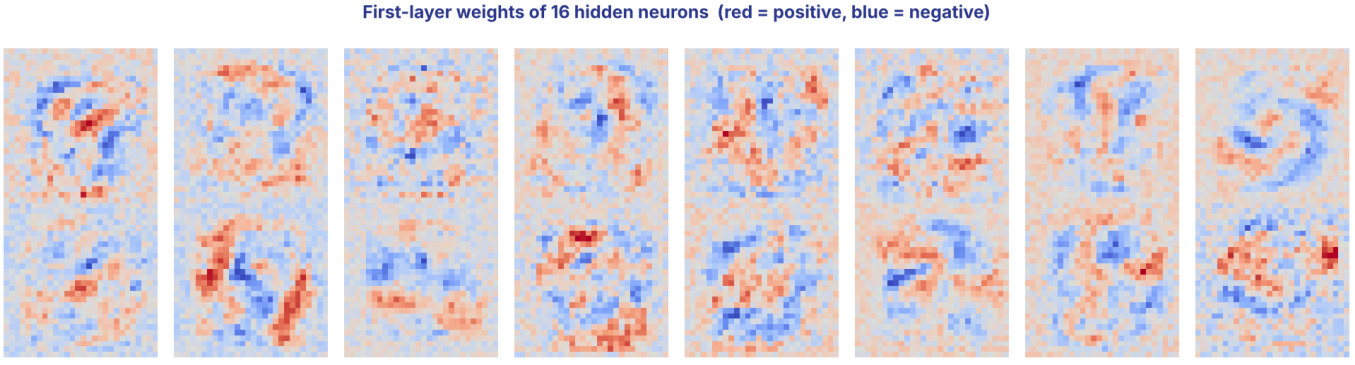




4 · How robust is the MLP? Move the digit by a few pixels

97–98 % sounds great. But what has the MLP actually learned? Every neuron in the first hidden layer has **one weight per pixel position**. Let's look at the 784 weights of a few neurons, drawn as 28 × 28 images:

```
W_first = mlp[1].weight.detach().cpu().reshape(-1, 28, 28)      # (256, 28, 28): one weight image per hidden neuron
fig, axes = plt.subplots(2, 8, figsize=(13, 3.6))
for k, ax in enumerate(axes.ravel()):
    lim = W_first[k].abs().max()
    ax.imshow(W_first[k], cmap="coolwarm", vmin=-lim, vmax=lim); ax.axis("off")
plt.suptitle("First-layer weights of 16 hidden neurons  (red = positive, blue = negative)", color=NAVY,
    fontweight="bold")
plt.tight_layout(); plt.show()
```



Each neuron is a template **glued to fixed pixel positions**. A stroke in the centre and the same stroke two pixels to the right hit completely different weights. MNIST digits are always centred in their 28 × 28 box, so the MLP never had to

deal with anything else. What happens if we move the test digits a little?

```
BACKGROUND = (0 - MEAN) / STD                    # the value of an empty pixel after standardising

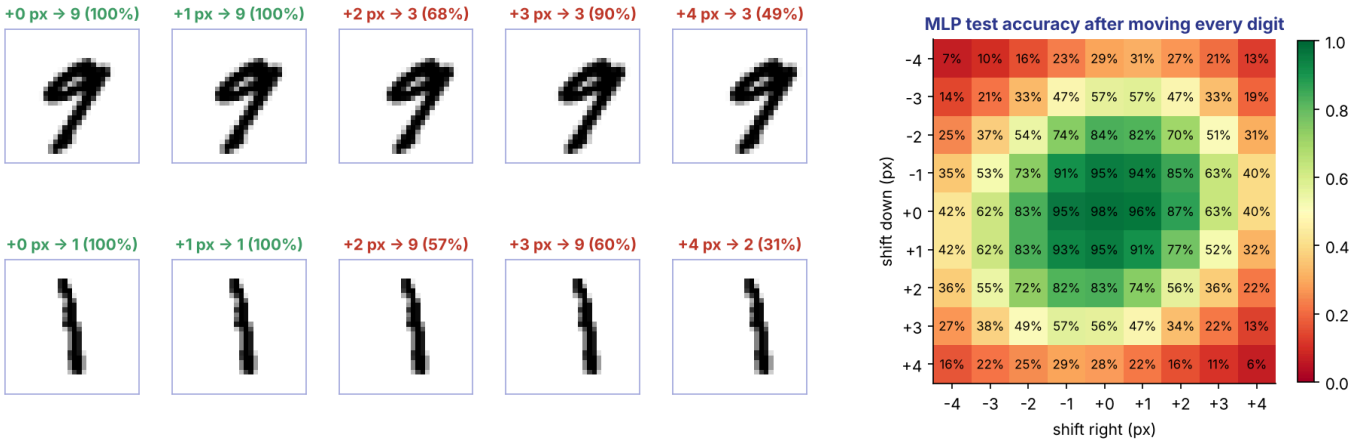
def shift(X, dx, dy):
    """Move images dx pixels to the right and dy pixels down; the uncovered border becomes background."""
    P = 8
    Xp = F.pad(X, (P, P, P, P), value=BACKGROUND)
    return Xp[:, P - dy:P - dy + 28, P - dx:P - dx + 28]

@torch.no_grad()
def accuracy(model, X, y):
    model.eval()
    pred = torch.cat([model(X[i:i + 2000].to(device)).argmax(1).cpu() for i in range(0, len(X), 2000)])
    return (pred == y).float().mean().item()

SHIFTS = list(range(-4, 5))
acc_mlp_shift = np.array([[accuracy(mlp, shift(X_test, dx, dy), y_test) for dx in SHIFTS] for dy in SHIFTS])
print("accuracy when moving all test digits sideways (dy = 0):")
print("  ".join(f"{dx:+d} px: {a:.0%}" for dx, a in zip(SHIFTS, acc_mlp_shift[SHIFTS.index(0)])))
```

```
accuracy when moving all test digits sideways (dy = 0):
-4 px: 42%  -3 px: 62%  -2 px: 83%  -1 px: 95%  +0 px: 98%  +1 px: 96%  +2 px: 87%  +3 px: 63%  +4 px: 40%
```

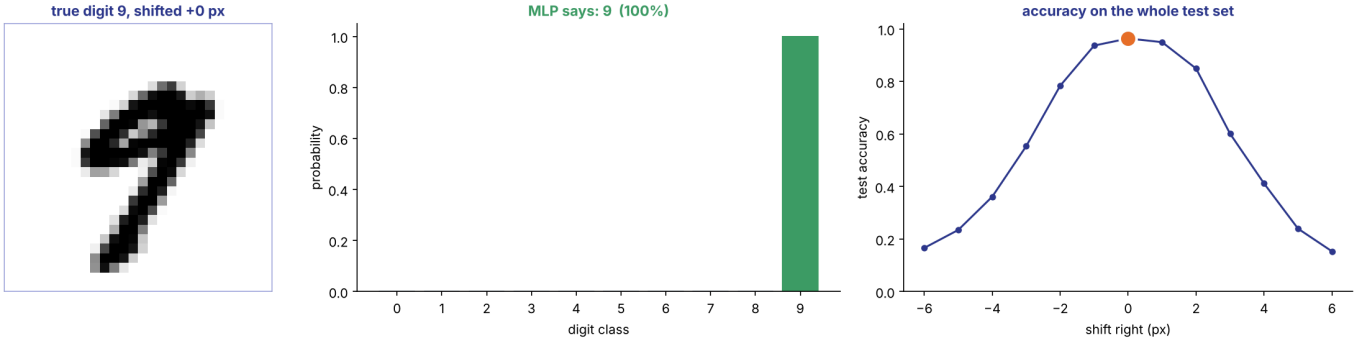
▸ 🎨 **Figure: shifted digits and the accuracy for every shift** *code hidden*



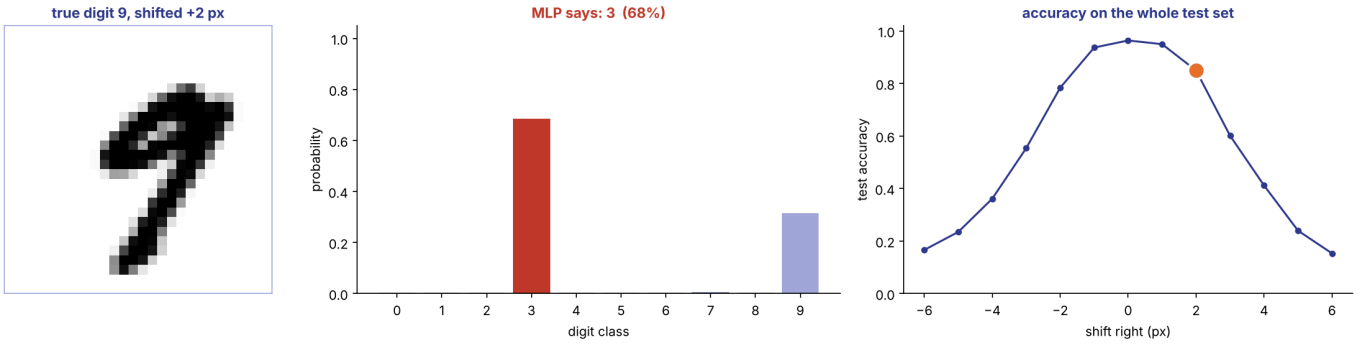
▸ 🎞️ **Animation: a digit slides sideways, the MLP changes its mind** *code hidden*

Animation · 25 frames

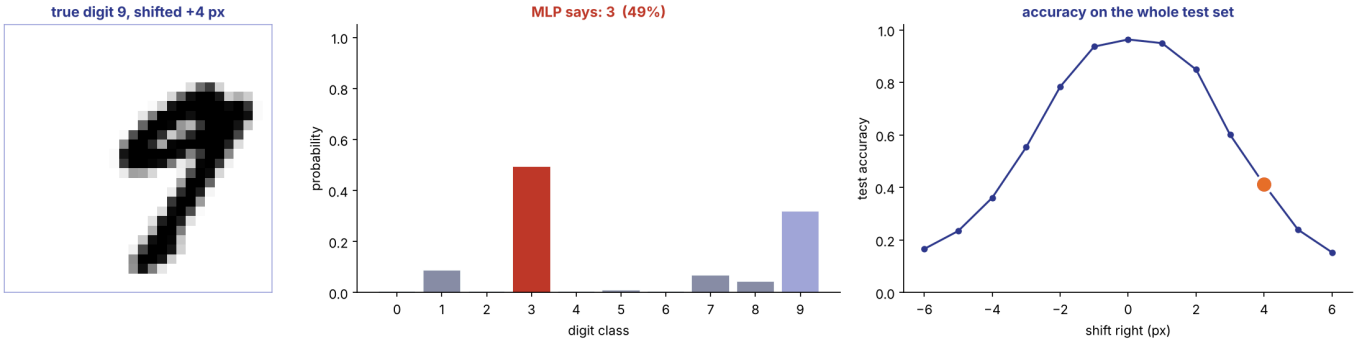
An interactive player in the notebook (play, pause, step frame by frame with ◀️ ▶️). Shown here: 3 still frames. Open in Colab to step through all of them.



Frame 1 of 25: in place: correct



Frame 3 of 25: shifted 2 px



Frame 5 of 25: shifted 4 px

A shift of 2–3 pixels, which no human would even notice, and the accuracy collapses. The MLP has no idea that a pattern means the same thing **wherever it appears**. We could show it shifted digits during training (*data augmentation*), but then it has to learn every digit at every position separately: more data, more parameters, longer training. In notebook 02 we repeat this experiment with a CNN, which has this knowledge built into its architecture.

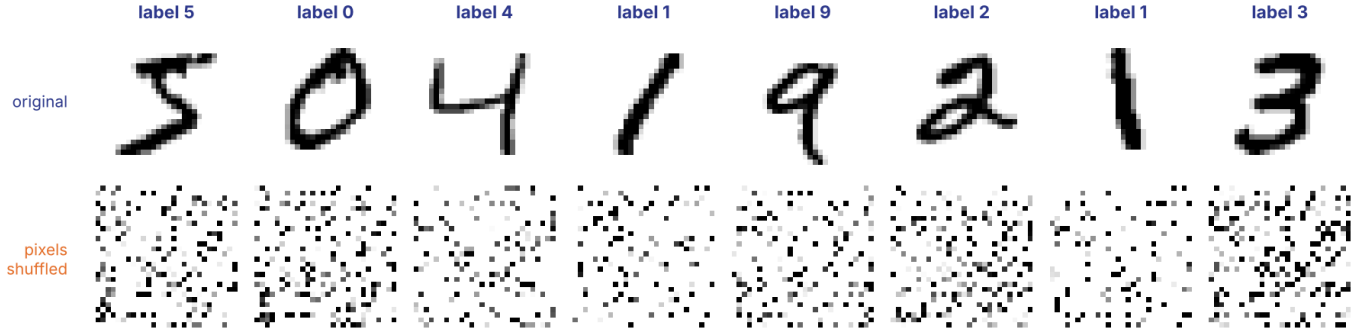
---

5 · Experiment: does the MLP care where the pixels are?

We apply the **same fixed random shuffle** of the 784 pixel positions to *every* image (train and test). For a human the digits become unrecognisable noise. What happens to the MLP?

```
torch.manual_seed(42)
perm = torch.randperm(784)
shuffle_pixels = lambda X: X.reshape(len(X), -1)[:, perm].reshape(-1, 28, 28)
X_train_perm, X_test_perm = shuffle_pixels(X_train), shuffle_pixels(X_test)

fig, axes = plt.subplots(2, 8, figsize=(12, 3.2))
for k in range(8):
    axes[0, k].imshow(X_train[k], cmap="gray_r"); axes[1, k].imshow(X_train_perm[k], cmap="gray_r")
    axes[0, k].set_title(f"label {y_train[k].item()}")
for ax in axes.ravel(): ax.axis("off")
axes[0, 0].text(-6, 14, "original", ha="right", va="center", color=NAVY); axes[1, 0].text(-6, 14, "pixels\nshuffled",
    ha="right", va="center", color=ORANGE)
plt.tight_layout(); plt.show()
```



```
torch.manual_seed(0)
mlp_perm = build_mlp()
hist_mlp_perm = fit(mlp_perm, *make_loaders(X_train_perm, y_train, X_test_perm, y_test), epochs=5)
print(f"\nMLP on original images: {hist_mlp['test_acc'][-1]:.2%}")
print(f"MLP on shuffled images: {hist_mlp_perm['test_acc'][-1]:.2%}")
```

```
epoch 1/5  train loss 0.2784  test loss 0.1411  test acc 95.58%  (10.4s)
epoch 2/5  train loss 0.1051  test loss 0.0838  test acc 97.25%  (1.5s)
epoch 3/5  train loss 0.0700  test loss 0.0787  test acc 97.43%  (1.9s)
epoch 4/5  train loss 0.0520  test loss 0.0852  test acc 97.39%  (1.4s)
epoch 5/5  train loss 0.0385  test loss 0.0796  test acc 97.56%  (1.4s)

MLP on original images: 97.52%
MLP on shuffled images: 97.56%
```

The MLP doesn't care! It gets (almost) the same accuracy on shuffled images.

A fully-connected layer treats its 784 inputs as an *unordered list*: shuffling the inputs just shuffles the rows of the first weight matrix, so it can learn the shuffled version equally well.

That sounds like a feature, but it is a **weakness**: the MLP does not use the most important prior knowledge we have about images:

- nearby pixels belong together (**locality**),
- a pattern (an edge, a hull, a "loop" of a 6) means the same thing wherever it appears (**translation**, see Section 4).

It has to learn everything from scratch, separately for every position. On 28 × 28 digits we can afford that. On real photos?

---

6 · From 28 × 28 to real photos: does the MLP scale?

MNIST images are tiny: 784 numbers. Our trawler at the standard input resolution of 224 × 224 × 3 has **150,528**, i.e. 192 × more. In an MLP every input is connected to every neuron of the first hidden layer, so the first layer grows with the **number of pixels**. Three ways to size the hidden layer:

- keep **256 neurons** like our MNIST MLP (a bottleneck: 150,528 numbers squeezed into 256 right away),
- **1000 neurons**, the number on the lecture slide,
- keep the **same shape** as our MNIST MLP (784 → 256, roughly one hidden neuron per three inputs).

Training needs about **16 bytes per parameter**: the weight, its gradient, and Adam's two running averages (4 bytes each).

```
def mlp_param_count(n_in, hidden):
    return n_in * hidden + hidden + hidden * 128 + 128 + 128 * 10 + 10        # n_in → hidden → 128 → 10

def available_memory():
    """Free memory for training on this machine in bytes: GPU memory, or RAM on a CPU runtime."""
    if device.type == "cuda": return torch.cuda.mem_get_info()[0]
    import psutil; return psutil.virtual_memory().available

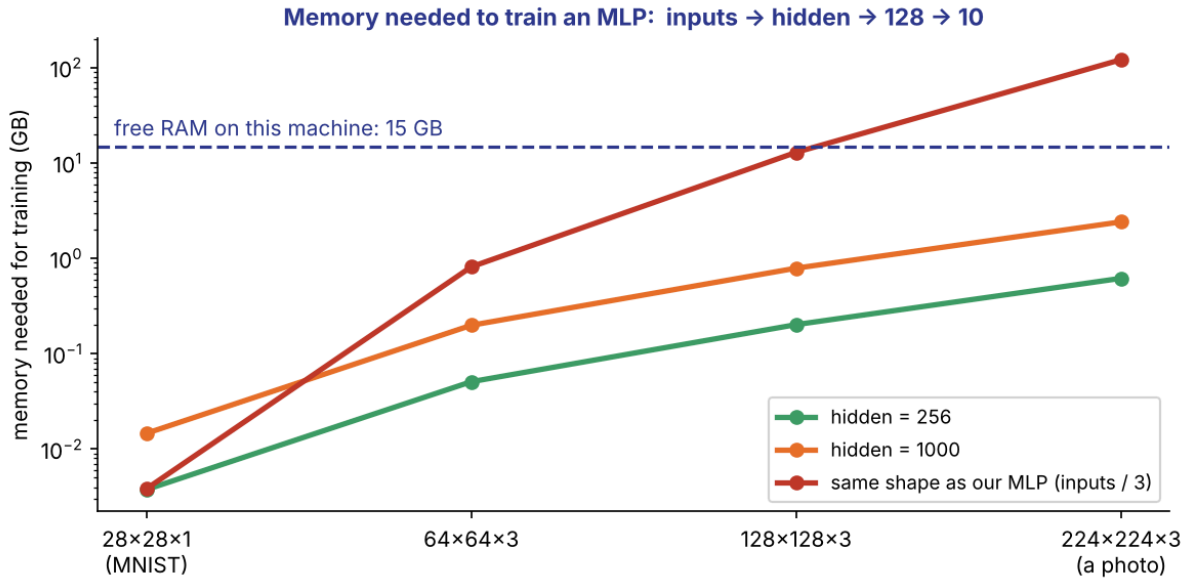
BYTES_PER_PARAM = 16
SIZES = {"28×28×1\n(MNIST)": 28 * 28, "64×64×3": 64 * 64 * 3, "128×128×3": 128 * 128 * 3, "224×224×3\n(a photo)": 224
    * 224 * 3}
WIDTHS = {"hidden = 256": lambda n: 256, "hidden = 1000": lambda n: 1000, "same shape as our MLP (inputs / 3)":
    lambda n: n // 3}

print(f"{'input':<11}" + "".join(f"{w.split(' (')[0]:>36}" for w in WIDTHS))
for name, n in SIZES.items():
    cols = [f"{mlp_param_count(n, f(n)):,} params ({mlp_param_count(n, f(n)) * BYTES_PER_PARAM / 1e9:.1f} GB)" for f
    in WIDTHS.values()]
    print(f"{name.splitlines()[0]:<11}" + "".join(f"{c:>36}" for c in cols))

mem = available_memory()
fig, ax = plt.subplots(figsize=(9, 4))
for (w, f), c in zip(WIDTHS.items(), [GREEN, ORANGE, RED]):
    ax.plot(range(len(SIZES)), [mlp_param_count(n, f(n)) * BYTES_PER_PARAM / 1e9 for n in SIZES.values()], "-o",
    color=c, lw=2.5, label=w)
ax.axhline(mem / 1e9, color=NAVY, ls="--", lw=1.5)
```

```
ax.text(-0.1, mem / 1e9 * 1.3, f"free {'GPU memory' if device.type == 'cuda' else 'RAM'} on this machine: {mem /
    1e9:.0f} GB", color=NAVY)
ax.set(yscale="log", xticks=range(len(SIZES)), ylabel="memory needed for training (GB)",
       title="Memory needed to train an MLP:  inputs → hidden → 128 → 10")
ax.set_xticklabels(SIZES.keys()); ax.legend(loc="lower right", fontsize=9); plt.show()
```

```
input                          hidden = 256                  hidden = 1000            same shape as our MLP
28×28×1              235,146 params (0.0 GB)          914,418 params (0.0 GB)          239,711 params (0.0 GB)
64×64×3            3,180,170 params (0.1 GB)       12,418,418 params (0.2 GB)       50,861,450 params (0.8 GB)
128×128×3         12,617,354 params (0.2 GB)       49,282,418 params (0.8 GB)      807,421,322 params (12.9 GB)
224×224×3         38,569,610 params (0.6 GB)      150,658,418 params (2.4 GB)    7,559,367,050 params (120.9 GB)
```



Even when it fits, it is slow. Let's **measure** one training step (forward + backward + Adam update, a mini-batch of 64 random images) for the MNIST MLP and for MLPs on 224 × 224 × 3 inputs, and extrapolate to a full epoch:

```
def sync():
    if device.type == "cuda": torch.cuda.synchronize()

def seconds_per_step(model, input_shape, batch=64, min_seconds=1.0):
    """Average wall-clock time of one training step on random data (at least 3 steps and 1 s, after a warm-up
    step)."""
    model.to(device); opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    x = torch.randn(batch, *input_shape, device=device); y = torch.randint(0, 10, (batch,), device=device)
    def step():
        loss = F.cross_entropy(model(x), y)
        opt.zero_grad(); loss.backward(); opt.step(); sync()
    step()                                                    # warm-up
    t0, n = time.time(), 0
    while n < 3 or time.time() - t0 < min_seconds:
        step(); n += 1
    return (time.time() - t0) / n

make_mlp = lambda n_in, hidden: nn.Sequential(nn.Flatten(), nn.Linear(n_in, hidden), nn.ReLU(),
                                              nn.Linear(hidden, 128), nn.ReLU(), nn.Linear(128, 10))
timings = {}
for name, shape, hidden in [("MNIST MLP   28×28×1,   hidden 256", (1, 28, 28), 256),
                            ("photo MLP  224×224×3,  hidden 256", (3, 224, 224), 256),
                            ("photo MLP  224×224×3,  hidden 1000", (3, 224, 224), 1000)]:
    n_params = mlp_param_count(int(np.prod(shape)), hidden)
    if n_params * BYTES_PER_PARAM > 0.6 * available_memory():
        print(f"{name}: skipped, does not fit into memory"); continue
    torch.manual_seed(0); model = make_mlp(int(np.prod(shape)), hidden)
    t = seconds_per_step(model, shape); timings[name] = t
    del model
    if device.type == "cuda": torch.cuda.empty_cache()
    epoch_60k, epoch_imagenet = t * 60_000 / 64, t * 1_281_167 / 64
    print(f"{name}: {n_params:>12,} params | {t * 1000:7.1f} ms per step | one epoch over 60k images: "
          f"{epoch_60k / 60:6.1f} min | over ImageNet (1.28 M images): {epoch_imagenet / 3600:5.1f} h")

base = timings.pop("MNIST MLP   28×28×1,   hidden 256")
for name, t in timings.items(): print(f"→ {name}: {t / base:.0f}× slower per step than the MNIST MLP")
```

```
MNIST MLP   28×28×1,   hidden 256:      235,146 params |     1.7 ms per step | one epoch over 60k images:    0.0 min | over ImageNet (1.28 M images):   0.0 h
photo MLP  224×224×3,  hidden 256:   38,569,610 params |   205.5 ms per step | one epoch over 60k images:    3.2 min | over ImageNet (1.28 M images):   1.1 h
photo MLP  224×224×3,  hidden 1000:  150,658,418 params |   835.8 ms per step | one epoch over 60k images:   13.1 min | over ImageNet (1.28 M images):   4.6 h
→ photo MLP  224×224×3,  hidden 256: 120× slower per step than the MNIST MLP
→ photo MLP  224×224×3,  hidden 1000: 487× slower per step than the MNIST MLP
```

And the "same shape as our MLP" version? It needs billions of parameters. Let's try to create just its **first layer**:

```python
n_in = 224 * 224 * 3; hidden = n_in // 3
n_params = mlp_param_count(n_in, hidden)
print(f"MLP {n_in:,} → {hidden:,} → 128 → 10: {n_params / 1e9:.1f} billion parameters")
print(f"  weights alone: {n_params * 4 / 1e9:.0f} GB · training with Adam: {n_params * BYTES_PER_PARAM / 1e9:.0f} GB
    . "
      f"free on this machine: {available_memory() / 1e9:.0f} GB")
if device.type == "cuda":
    try:
        first_layer = nn.Linear(n_in, hidden, device=device)
        print("…it fitted?! You have a very big GPU.")
        del first_layer
    except torch.cuda.OutOfMemoryError as e:
        print("\n❌ torch.cuda.OutOfMemoryError:", ". ".join(str(e).split(". ")[:2]))
    torch.cuda.empty_cache()
else:
    print("\n❌ Not even trying on a CPU runtime: allocating this much RAM would crash the notebook.")
```

```
MLP 150,528 → 50,176 → 128 → 10: 7.6 billion parameters
  weights alone: 30 GB · training with Adam: 121 GB · free on this machine: 15 GB

❌ Not even trying on a CPU runtime: allocating this much RAM would crash the notebook.
```

MLPs do not scale to real images.

- The number of weights grows with the number of pixels: 192 × more inputs means 192 × more first-layer weights.
- Training gets **slower** by orders of magnitude, even for a modest hidden layer (see the measurement above).
- An MLP with the same shape as our MNIST network **does not even fit into memory**.
- And even if it did: billions of parameters, each tied to one pixel position, would need an enormous amount of data to learn every pattern at every position (Section 4).

We need a layer that **reuses** the same few weights at every position of the image: the **convolution**.

Next → 02 · Convolutional Neural Networks: networks built for images. We will repeat the shift and the shuffle experiments there.

Summary

1. To a computer an image is a $H \times W \times C$ array of numbers, and raw pixel distance says little about content.
2. Core CV tasks: **classification**, **detection**, **segmentation**, **depth estimation**; all need good spatial features.
3. An MLP on flattened MNIST reaches ~97.5 % test accuracy with 235 k parameters.
4. But it is **fragile**: its weights are glued to pixel positions, so shifting a digit by 2–3 pixels breaks it.
5. It is **blind to spatial structure**: shuffling the pixels doesn't hurt it.
6. It **does not scale**: at 224 × 224 × 3 the first layer alone has tens of millions to billions of weights, training slows down by orders of magnitude, and the full-size version does not fit into memory. CNNs fix all three.